udemy - Security

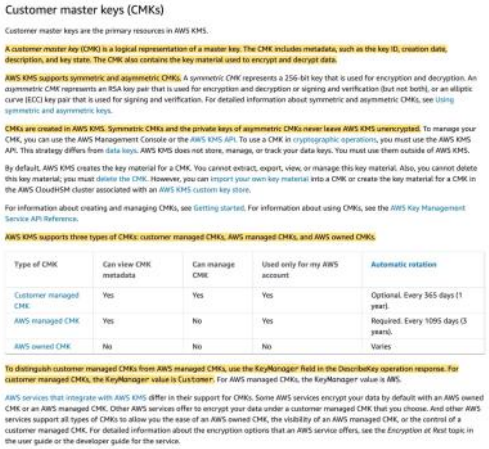
Monday, December 8, 2025 10:04 AM

Question 1

Which of the following best describes how KMS Encryption works?

- KMS receives CMK from the client at every Encrypt call, and encrypts the data with that
- KMS stores the CMK, and receives data from the clients, which it encrypts and sends back
- KMS generates a new CMK for each Encrypt call and encrypts the data with it
- KMS sends the CMK to the client, which performs the encryption and then deletes the CMK

Overall explanation
Correct option:
KMS stores the CMK, and receives data from the clients, which it encrypts and sends back
A customer master key (CMK) is a logical representation of a master key. The CMK includes metadata, such as the key ID, creation date, description, and key state. The CMK also contains the key material used to encrypt and decrypt data. You can generate CMKs in KMS, in an AWS CloudHSM cluster, or import them from your key management infrastructure.
AWS KMS supports symmetric and asymmetric CMKs. A symmetric CMK represents a 256-bit key that is used for encryption and decryption. An asymmetric CMK represents an RSA key pair that is used for encryption and decryption or signing and verification (but not both), or an elliptic curve (ECC) key pair that is used for signing and verification.
AWS KMS supports three types of CMKs: customer-managed CMKs, AWS managed CMKs, and AWS owned CMKs.
Overview of Customer master keys (CMKs):



Customer master keys (CMKs)

Customer master keys are the primary resources in AWS KMS.

A customer master key (CMK) is a logical representation of a master key. The CMK includes metadata, such as the key ID, creation date, description, and key state. The CMK also contains the key material used to encrypt and decrypt data.

AWS KMS supports symmetric and asymmetric CMKs. A symmetric CMK represents a 256-bit key that is used for encryption and decryption. An asymmetric CMK represents an RSA key pair that is used for encryption and decryption or signing and verification (but not both), or an elliptic curve (ECC) key pair that is used for signing and verification. For detailed information about symmetric and asymmetric CMKs, see Using symmetric and asymmetric keys.

CMKs are created in AWS KMS. Symmetric CMKs and the private keys of asymmetric CMKs never leave AWS KMS unencrypted. To manage your CMK, you can use the AWS Management Console or the AWS KMS API. To use a CMK in cryptographic operations, you must use the AWS KMS API. This strategy differs from data keys. AWS KMS does not store, manage, or track your data keys. You must use them outside of AWS KMS.

By default, AWS KMS creates the key material for a CMK. You cannot extract, export, view, or manage this key material. Also, you cannot delete this key material; you must delete the CMK. However, you can import your own key material into a CMK or create the key material for a CMK in the AWS CloudHSM cluster associated with an AWS KMS custom key store.

For information about creating and managing CMKs, see Getting started. For information about using CMKs, see the AWS Key Management Service API Reference.

AWS KMS supports three types of CMKs: customer managed CMKs, AWS managed CMKs, and AWS owned CMKs.

Type of CMK	Can view CMK metadata	Can manage CMK	Used only for my AWS account	Automatic rotation
Customer managed CMK	Yes	Yes	Yes	Optional. Every 365 days (1 year).
AWS managed CMK	Yes	No	Yes	Required. Every 1095 days (3 years).
AWS owned CMK	No	No	No	Varies

To distinguish customer managed CMKs from AWS managed CMKs, use the KeyManager field in the DescribeKey operation response. For customer managed CMKs, the KeyManager value is Customer. For AWS managed CMKs, the KeyManager value is AWS.

AWS services that integrate with AWS KMS differ in their support for CMKs. Some AWS services encrypt your data by default with an AWS owned CMK or an AWS managed CMK. Other AWS services offer to encrypt your data under a customer managed CMK that you choose. And other AWS services support all types of CMKs to allow you the ease of an AWS owned CMK, the visibility of an AWS managed CMK, or the control of a customer managed CMK. For detailed information about the encryption options that an AWS service offers, see the Encryption at Rest topic in the user guide or the developer guide for the service.

via - https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html#master_keys
Incorrect options:
KMS receives CMK from the client at every encrypt call, and encrypts the data with that - You can import your own CMK (Customer Master Key) but it is done once and then you can encrypt/decrypt as needed.
KMS sends the CMK to the client, which performs the encryption and then deletes the CMK - KMS does not send CMK to the client, KMS itself encrypts, and then decrypts the data.
KMS generates a new CMK for each Encrypt call and encrypts the data with it - KMS does not generate a new key each time but you can have KMS rotate the keys for you. Best practices discourage extensive reuse of encryption keys so it is good practice to generate new keys.

Question 2

The manager at an IT company wants to set up member access to user-specific folders in an Amazon S3 bucket - bucket-a. So, user x can only access files in his folder - bucket-a/user/user-x/ and user y can only access files in her folder - bucket-a/user/user-y/ and so on.
As a Developer Associate, which of the following IAM constructs would you recommend so that the policy snippet can be made generic for all team members and the manager does not need to create separate IAM policy for each team member?

- IAM policy condition
- IAM policy resource
- IAM policy variables
- IAM policy principal

Overall explanation
Correct option:
IAM policy variables
Instead of creating individual policies for each user, you can use policy variables and create a single policy that applies to multiple users (a group policy). Policy variables act as placeholders. When you make a request to AWS, the placeholder is replaced by a value from the request when the policy is evaluated.
As an example, the following policy gives each of the users in the group full programmatic access to a user-specific object (their own "home directory") in Amazon S3.

In some cases, you might not know the exact name of the resource when you write the policy. You might want to generalize the policy so it works for many users without having to make a unique copy of the policy for each user. For example, consider writing a policy to allow each user to have access to his or her own objects in an Amazon S3 bucket, as in the previous example. But don't create a separate policy for each user that explicitly specifies the user's name as part of the resource. Instead, create a single group policy that works for any user in that group.

You can do this by using policy variables, a feature that lets you specify placeholders in a policy. When the policy is evaluated, the policy variables are replaced with values that come from the context of the request itself.

The following example shows a policy for an Amazon S3 bucket that uses a policy variable.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": ["s3:ListBucket"],
      "Effect": "Allow",
      "Resource": ["arn:aws:s3:::mybucket"],
      "Condition": {"StringLike": {"s3:prefix": ["${aws:username}/*"]}}
    },
    {
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Effect": "Allow",
      "Resource": ["arn:aws:s3:::mybucket/${aws:username}/*"]
    }
  ]
}
```

When this policy is evaluated, IAM replaces the variable ${aws:username} with the friendly name of the actual current user. This means that a single policy applied to a group of users can control access to a bucket by using the user name as part of the resource's name.

The variable is marked using a $ prefix followed by a pair of curly braces ({ }). Inside the ${ } characters, you can include the name of the value from the request that you want to use in the policy. The values you can use are discussed later on this page.

Note
In order to use policy variables, you must include the Version element in a statement, and the version must be set to a version that supports policy variables. Variables were introduced in version 2012-10-17. Earlier versions of the policy language don't support policy variables. If you don't include the Version element and set it to an appropriate version date, variables like ${aws:username} are treated as literal strings in the policy.

via - https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_variables.html
Incorrect options:
IAM policy principal - You can use the Principal element in a policy to specify the principal that is allowed or denied access to a resource (In IAM, a principal is a person or application that can make a request for an action or operation on an AWS resource. The principal is authenticated as the AWS account root user or an IAM entity to make requests to AWS). You cannot use the Principal element in an IAM identity-based policy. You can use it in the trust policies for IAM roles and in resource-based policies.
IAM policy condition - The Condition element (or Condition block) lets you specify conditions for when a policy is in effect, like so - "Condition" : { "StringEquals" : { "aws:username" : "johndoe" }}. This can not be used to address the requirements of the given use-case.
IAM policy resource - The Resource element specifies the object or objects that the statement covers. You specify a resource using an ARN. This can not be used to address the requirements of the

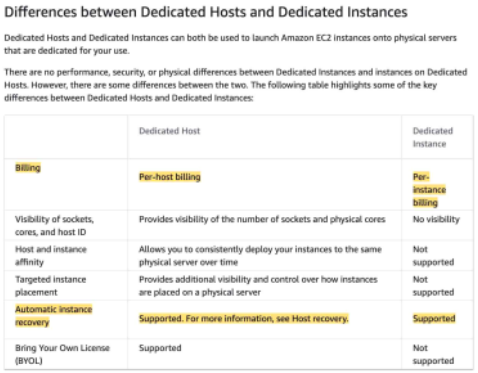
given use-case.

Question 3

A cybersecurity firm wants to run their applications on single-tenant hardware to meet security guidelines.
Which of the following is the MOST cost-effective way of isolating their Amazon EC2 instances to a single tenant?

- Dedicated Hosts
- On-Demand Instances
- Spot Instances
- Dedicated Instances

Overall explanation
Correct option:
Dedicated Instances - Dedicated Instances are Amazon EC2 instances that run in a virtual private cloud (VPC) on hardware that's dedicated to a single customer. Dedicated Instances that belong to different AWS accounts are physically isolated at a hardware level, even if those accounts are linked to a single-payer account. However, Dedicated Instances may share hardware with other instances from the same AWS account that are not Dedicated Instances.
A Dedicated Host is also a physical server that's dedicated for your use. With a Dedicated Host, you have visibility and control over how instances are placed on the server.
Differences between Dedicated Hosts and Dedicated Instances:



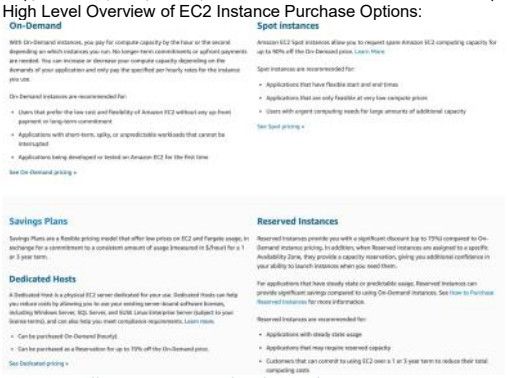
Differences between Dedicated Hosts and Dedicated Instances

Dedicated Hosts and Dedicated Instances can both be used to launch Amazon EC2 instances onto physical servers that are dedicated for your use.

There are no performance, security, or physical differences between Dedicated Instances and instances on Dedicated Hosts. However, there are some differences between the two. The following table highlights some of the key differences between Dedicated Hosts and Dedicated Instances:

	Dedicated Host	Dedicated Instance
Billing	Per-host billing	Per-instance billing
Visibility of sockets, cores, and host ID	Provides visibility of the number of sockets and physical cores	No visibility
Host and instance affinity	Allows you to consistently deploy your instances to the same physical server over time	Not supported
Targeted instance placement	Provides additional visibility and control over how instances are placed on a physical server	Not supported
Automatic instance recovery	Supported. For more information, see Host recovery.	Supported
Bring Your Own License (BYOL)	Supported	Not supported

via - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/dedicated-hosts-overview.html#dedicated-hosts-dedicated-instances
Incorrect options:
Spot Instances - A Spot Instance is an unused EC2 instance that is available for less than the On-Demand price. Your Spot Instance runs whenever capacity is available and the maximum price per hour for your request exceeds the Spot price. Any instance present with unused capacity will be allocated. Even though this is cost-effective, it does not fulfill the single-tenant hardware requirement of the client and hence is not the correct option.
Dedicated Hosts - An Amazon EC2 Dedicated Host is a physical server with EC2 instance capacity fully dedicated to your use. Dedicated Hosts allow you to use your existing software licenses on EC2 instances. With a Dedicated Host, you have visibility and control over how instances are placed on the server. This option is costlier than the Dedicated Instance and hence is not the right choice for the current requirement.
On-Demand Instances - With On-Demand Instances, you pay for compute capacity by the second with no long-term commitments. You have full control over its lifecycle—you decide when to launch, stop, hibernate, start, reboot, or terminate it. Hardware isolation is not possible and on-demand has one of the costliest instance charges and hence is not the correct answer for current requirements.
High Level Overview of EC2 Instance Purchase Options:



On-Demand

Spot instances

Savings Plans

Reserved Instances

Dedicated Hosts

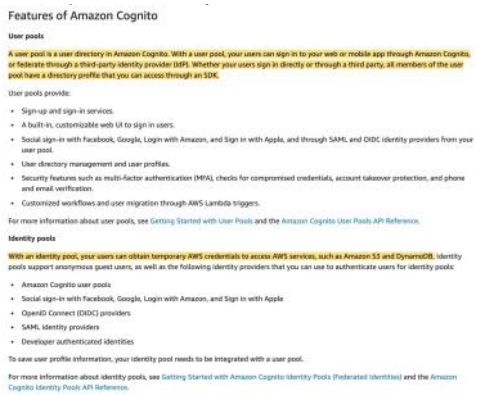
via - https://aws.amazon.com/ec2/pricing/

Question 4

Which of the following security credentials can only be created by the AWS Account root user?

- CloudFront Key Pairs
- IAM User Access Keys
- EC2 Instance Key Pairs
- IAM User passwords

Overall explanation
Correct option:
For Amazon CloudFront, you use key pairs to create signed URLs for private content, such as when you want to distribute restricted content that someone paid for.
CloudFront Key Pairs - IAM users can't create CloudFront key pairs. You must log in using root credentials to create key pairs.
To create signed URLs or signed cookies, you need a signer. A signer is either a trusted key group that you create in CloudFront, or an AWS account that contains a CloudFront key pair. AWS recommends that you use trusted key groups with signed URLs and signed cookies instead of using CloudFront key pairs.
Incorrect options:
EC2 Instance Key Pairs - You use key pairs to access Amazon EC2 instances, such as when you use SSH to log in to a Linux instance. These key pairs can be created from the IAM user login and do not need root user access.
IAM User Access Keys - Access keys consist of two parts: an access key ID and a secret access key. You use access keys to sign programmatic requests that you make to AWS if you use AWS CLI commands (using the SDKs) or using AWS API operations. IAM users can create their own Access Keys, does not need root access.
IAM User passwords - Every IAM user has access to his own credentials and can reset the password whenever they need to.



Features of Amazon Cognito

User pools

A user pool is a user directory in Amazon Cognito. With a user pool, your users can sign in to your web or mobile app through Amazon Cognito, or federate through a third-party identity provider (IdP). Whether your users sign in directly or through a third party, all members of the user pool have a directory profile that you can access through an SDK.

User pools provide:

- Sign-up and sign-in services.
- A built-in, customizable web UI to sign in users.
- Social sign-in with Facebook, Google, Login with Amazon, and Sign in with Apple, and through SAML and OIDC identity providers from your user pool.
- User directory management and user profiles.
- Security features such as multi-factor authentication (MFA), checks for compromised credentials, account takeover protection, and phone and email verification.
- Customized workflows and user migration through AWS Lambda triggers.

For more information about user pools, see Getting Started with User Pools and the Amazon Cognito User Pools API Reference.

Identity pools

With an identity pool, your users can obtain temporary AWS credentials to access AWS services, such as Amazon S3 and DynamoDB. Identity pools support anonymous guest users, as well as the following identity providers that you can use to authenticate users for identity pools:

- Amazon Cognito user pools
- Social sign-in with Facebook, Google, Login with Amazon, and Sign in with Apple
- OpenID Connect (OIDC) providers
- SAML identity providers
- Developer authenticated identities

To save user profile information, your identity pool needs to be integrated with a user pool.

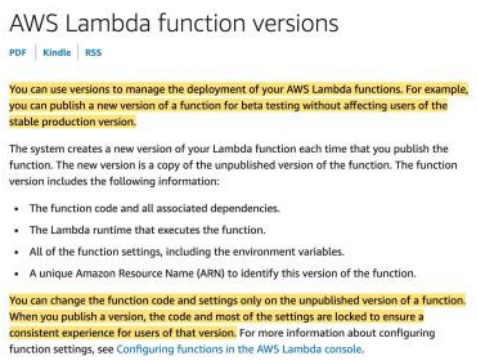
For more information about identity pools, see Getting Started with Amazon Cognito Identity Pools (Federated Identities) and the Amazon Cognito Identity Pools API Reference.

Question 5

A development team at a social media company uses AWS Lambda functions for its serverless stack on AWS Cloud. For a new deployment, the Team Lead wants to send only a certain portion of the traffic to a new version of a Lambda function. In case the deployment goes wrong, the solution should also support the ability to roll back to a previous version of the Lambda function, with MIMINUM downtime for the application.
As a Developer Associate, which of the following options would you recommend to address this use-case?

- Set up the application to use an alias that points to the current version. Deploy the new version of the code and configure the alias to send 10% of the users to this new version. If the deployment goes wrong, reset the alias to point all traffic to the current version
- Set up the application to have multiple alias of the Lambda function. Deploy the new version of the code. Configure a new alias that points to the current alias of the Lambda function for handling 10% of the traffic. If the deployment goes wrong, reset the new alias to point all traffic to the most recent working alias of the Lambda function
- Set up the application to directly deploy the new Lambda version. If the deployment goes wrong, reset the application back to the current version using the version number in the ARN
- Set up the application to use an alias that points to the current version. Deploy the new version of the code and configure alias to send all users to this new version. If the deployment goes wrong, reset the alias to point to the current version

Overall explanation
Correct option:
Set up the application to use an alias that points to the current version. Deploy the new version of the code and configure the alias to send 10% of the users to this new version. If the deployment goes wrong, reset the alias to point all traffic to the current version
You can use versions to manage the deployment of your AWS Lambda functions. For example, you can publish a new version of a function for beta testing without affecting users of the stable production version. You can change the function code and settings only on the unpublished version of a function. When you publish a version, the code and most of the settings are locked to ensure a consistent experience for users of that version.
You can create one or more aliases for your AWS Lambda function. A Lambda alias is like a pointer to a specific Lambda function version. You can use routing configuration on an alias to send a portion of traffic to a Lambda function version. For example, you can reduce the risk of deploying a new version by configuring the alias to send most of the traffic to the existing version, and only a small percentage of traffic to the new version.



AWS Lambda function versions

PDF | Kindle | RSS

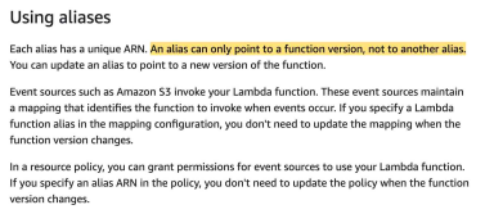
You can use versions to manage the deployment of your AWS Lambda functions. For example, you can publish a new version of a function for beta testing without affecting users of the stable production version.

The system creates a new version of your Lambda function each time that you publish the function. The new version is a copy of the unpublished version of the function. The function version includes the following information:

- The function code and all associated dependencies.
- The Lambda runtime that executes the function.
- All of the function settings, including the environment variables.
- A unique Amazon Resource Name (ARN) to identify this version of the function.

You can change the function code and settings only on the unpublished version of a function. When you publish a version, the code and most of the settings are locked to ensure a consistent experience for users of that version. For more information about configuring function settings, see Configuring functions in the AWS Lambda console.

via - https://docs.aws.amazon.com/lambda/latest/dg/configuration-versions.html



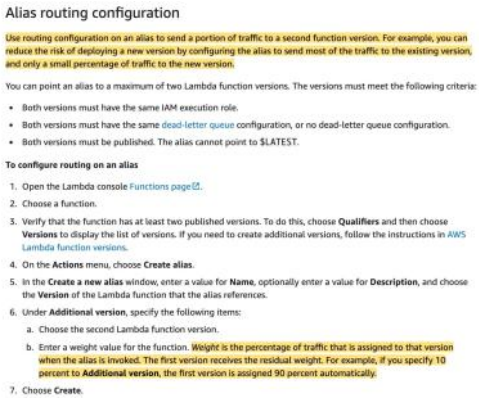
Using aliases

Each alias has a unique ARN. An alias can only point to a function version, not to another alias. You can update an alias to point to a new version of the function.

Event sources such as Amazon S3 invoke your Lambda function. These event sources maintain a mapping that identifies the function to invoke when events occur. If you specify a Lambda function alias in the mapping configuration, you don't need to update the mapping when the function version changes.

In a resource policy, you can grant permissions for event sources to use your Lambda function. If you specify an alias ARN in the policy, you don't need to update the policy when the function version changes.

via - https://docs.aws.amazon.com/lambda/latest/dg/configuration-aliases.html



Alias routing configuration

Use routing configuration on an alias to send a portion of traffic to a second function version. For example, you can reduce the risk of deploying a new version by configuring the alias to send most of the traffic to the existing version, and only a small percentage of traffic to the new version.

You can point an alias to a maximum of two Lambda function versions. The versions must meet the following criteria:

- Both versions must have the same IAM execution role.
- Both versions must have the same dead-letter queue configuration, or no dead-letter queue configuration.
- Both versions must be published. The alias cannot point to $LATEST.

To configure routing on an alias

1. Open the Lambda console Functions page.
2. Choose a function.
3. Verify that the function has at least two published versions. To do this, choose **Qualifiers** and then choose **Versions** to display the list of versions. If you need to create additional versions, follow the instructions in AWS Lambda function versions.
4. On the **Actions** menu, choose **Create alias**.
5. In the **Create a new alias** window, enter a value for **Name**, optionally enter a value for **Description**, and choose the **Version** of the Lambda function that the alias references.
6. Under **Additional version**, specify the following items:
   a. Choose the second Lambda function version.
   b. Enter a weight value for the function. Weight is the percentage of traffic that is assigned to that version when the alias is invoked. The first version receives the residual weight. For example, if you specify 10 percent to **Additional version**, the first version is assigned 90 percent automatically.
7. Choose **Create**.

via - https://docs.aws.amazon.com/lambda/latest/dg/configuration-aliases.html
Incorrect options:
Set up the application to use an alias that points to the current version. Deploy the new version of the code and configure alias to send all users to this new version. If the deployment

goes wrong, reset the alias to point to the current version - In this case, the application uses an alias to send all traffic to the new version which does not meet the requirement of sending only a certain portion of the traffic to the new Lambda version. In addition, if the deployment goes wrong, the application would see a downtime. Hence this option is incorrect.
Set up the application to directly deploy the new Lambda version. If the deployment goes wrong, reset the application back to the current version using the version number in the ARN - In this case, the application sends all traffic to the new version which does not meet the requirement of sending only a certain portion of the traffic to the new Lambda version. In addition, if the deployment goes wrong, the application would see a downtime. Hence this option is incorrect.
Set up the application to have multiple alias of the Lambda function. Deploy the new version of the code. Configure a new alias that points to the current alias of the Lambda function for handling 10% of the traffic. If the deployment goes wrong, reset the new alias to point all traffic to the most recent working alias of the Lambda function - This option has been added as a distractor. The alias for a Lambda function can only point to a Lambda function version. It cannot point to another alias.

Question 6

A developer is configuring a bucket policy that denies upload object permission to any requests that do not include the x-amz-server-side-encryption header requesting server-side encryption with SSE-KMS for an Amazon S3 bucket - examplebucket.
Which of the following policies is the right fit for the given requirement?
Your answer is incorrect

```
{
  "Version":"2012-10-17",
  "Id":"PutObjectPolicy",
  "Statement":[{
      "Sid":"DenyUnEncryptedObjectUploads",
      "Effect":"Deny",
      "Principal":"*",
      "Action":"s3:PutObject",
      "Resource":"arn:aws:s3:::examplebucket/*",
      "Condition":{
        "StringNotEquals":{
          "s3:x-amz-server-side-encryption":"false"
        }
      }
    }
  ]
}
```

```
{
  "Version":"2012-10-17",
  "Id":"PutObjectPolicy",
  "Statement":[{
      "Sid":"DenyUnEncryptedObjectUploads",
      "Effect":"Deny",
      "Principal":"*",
      "Action":"s3:GetObject",
      "Resource":"arn:aws:s3:::examplebucket/*",
      "Condition":{
        "StringNotEquals":{
          "s3:x-amz-server-side-encryption":"aws:AES256"
        }
      }
    }
  ]
}
```

Correct answer

```
{
  "Version":"2012-10-17",
  "Id":"PutObjectPolicy",
  "Statement":[{
      "Sid":"DenyUnEncryptedObjectUploads",
      "Effect":"Deny",
      "Principal":"*",
      "Action":"s3:PutObject",
      "Resource":"arn:aws:s3:::examplebucket/*",
      "Condition":{
        "StringNotEquals":{
          "s3:x-amz-server-side-encryption":"aws:kms"
        }
      }
    }
  ]
}
```

```
{
  "Version":"2012-10-17",
  "Id":"PutObjectPolicy",
  "Statement":[{
      "Sid":"DenyUnEncryptedObjectUploads",
      "Effect":"Deny",
      "Principal":"*",
      "Action":"s3:PutObject",
      "Resource":"arn:aws:s3:::examplebucket/*",
      "Condition":{
        "StringEquals":{
          "s3:x-amz-server-side-encryption":"aws:kms"
        }
      }
    }
  ]
}
```

Overall explanation
Correct option:
{ "Version":"2012-10-17", "Id":"PutObjectPolicy", "Statement":[{ "Sid":"DenyUnEncryptedObjectUploads", "Effect":"Deny", "Principal":"*", "Action":"s3:PutObject", "Resource":"arn:aws:s3:::examplebucket/*", "Condition":{ "StringNotEquals":{ "s3:x-amz-server-side-encryption":"aws:kms" } } }] } - This bucket policy denies upload object (s3:PutObject) permission if the request does not include the x-amz-server-side-encryption header requesting server-side encryption with SSE-KMS. To ensure that a particular AWS KMS CMK be used to encrypt the objects in a bucket, you can use the s3:x-amz-server-side-encryption-aws-kms-key-id condition key. To specify the AWS KMS CMK, you must use a key Amazon Resource Name (ARN) that is in the "arn:aws:kms:region:acct-id:key/key-id" format.
When you upload an object, you can specify the AWS KMS CMK using the x-amz-server-side-encryption-aws-kms-key-id header. If the header is not present in the request, Amazon S3 assumes the AWS-managed CMK.
Incorrect options:
{ "Version":"2012-10-17", "Id":"PutObjectPolicy", "Statement":[{ "Sid":"DenyUnEncryptedObjectUploads", "Effect":"Deny", "Principal":"*", "Action":"s3:PutObject", "Resource":"arn:aws:s3:::examplebucket/*", "Condition":{ "StringEquals":{ "s3:x-amz-server-side-encryption":"aws:kms" } } }] } - The condition is incorrect in this policy. The condition should use StringNotEquals.
{ "Version":"2012-10-17", "Id":"PutObjectPolicy", "Statement":[{ "Sid":"DenyUnEncryptedObjectUploads", "Effect":"Deny", "Principal":"*", "Action":"s3:GetObject", "Resource":"arn:aws:s3:::examplebucket/*", "Condition":{ "StringNotEquals":{ "s3:x-amz-server-side-encryption":"aws:AES256" } } }] } - AES256 is used for Amazon S3-managed encryption keys (SSE-S3). Amazon S3 server-side encryption uses one of the strongest block ciphers available to encrypt your data, 256-bit Advanced Encryption Standard (AES-256).
{ "Version":"2012-10-17", "Id":"PutObjectPolicy", "Statement":[{ "Sid":"DenyUnEncryptedObjectUploads", "Effect":"Deny", "Principal":"*", "Action":"s3:PutObject", "Resource":"arn:aws:s3:::examplebucket/*", "Condition":{ "StringNotEquals":{ "s3:x-amz-server-side-encryption":"false" } } }] } - The condition is incorrect in this policy. The condition should use "s3:x-amz-server-side-encryption":"aws:kms".

Question 7

A development team has configured inbound traffic for the relevant ports in both the Security Group of the EC2 instance as well as the Network Access Control List (NACL) of the subnet for the EC2 instance. The team is, however, unable to connect to the service running on the Amazon EC2 instance.
As a developer associate, which of the following will you recommend to fix this issue?

- Network ACLs are stateful, so allowing inbound traffic to the necessary ports enables the connection. Security Groups are stateless, so you must allow both inbound and outbound traffic
- Rules associated with Network ACLs should never be modified from the command line. An attempt to modify rules from the command line blocks the rule and results in an erratic behavior
- Security Groups are stateful, so allowing inbound traffic to the necessary ports enables the connection. Network ACLs are stateless, so you must allow both inbound and outbound traffic
- IAM Role defined in the Security Group is different from the IAM Role that is given access in the Network ACLs

Overall explanation
Correct option:
Security Groups are stateful, so allowing inbound traffic to the necessary ports enables the connection. Network ACLs are stateless, so you must allow both inbound and outbound traffic - Security groups are stateful, so allowing inbound traffic to the necessary ports enables the connection. Network ACLs are stateless, so you must allow both inbound and outbound traffic.
To enable the connection to a service running on an instance, the associated network ACL must allow both inbound traffic on the port that the service is listening on as well as allow outbound traffic from ephemeral ports. When a client connects to a server, a random port from the ephemeral port range (1024-65535) becomes the client's source port.
The designated ephemeral port then becomes the destination port for return traffic from the service, so outbound traffic from the ephemeral port must be allowed in the network ACL.
By default, network ACLs allow all inbound and outbound traffic. If your network ACL is more restrictive, then you need to explicitly allow traffic from the ephemeral port range.
If you accept traffic from the internet, then you also must establish a route through an internet gateway. If you accept traffic over VPN or AWS Direct Connect, then you must establish a route through a virtual private gateway.
Incorrect options:
Network ACLs are stateful, so allowing inbound traffic to the necessary ports enables the connection. Security Groups are stateless, so you must allow both inbound and outbound traffic - This is incorrect as already discussed.
IAM Role defined in the Security Group is different from the IAM Role that is given access in the Network ACLs - This is a made-up option and just added as a distractor.
Rules associated with Network ACLs should never be modified from the command line. An attempt to modify rules from the command line blocks the rule and results in an erratic behavior - This option is a distractor. AWS does not support modifying rules of Network ACLs from the command line tool.

Question 8

A Developer has been entrusted with the job of securing certain S3 buckets that are shared by a large team of users. Last time, a bucket policy was changed, the bucket was erroneously available for everyone, outside the organization too.
Which feature/service will help the developer identify similar security issues with minimum effort?

- Access Advisor feature on IAM console
- S3 Analytics
- S3 Object Lock
- IAM Access Analyzer

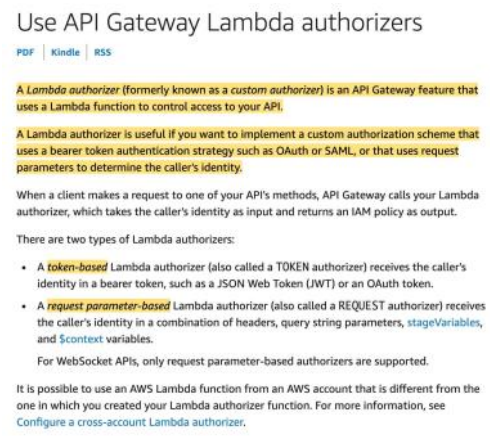
Overall explanation
Correct option:
IAM Access Analyzer - AWS IAM Access Analyzer helps you identify the resources in your organization and accounts, such as Amazon S3 buckets or IAM roles, that are shared with an external entity. This lets you identify unintended access to your resources and data, which is a security risk.
You can set the scope for the analyzer to an organization or an AWS account. This is your zone of trust. The analyzer scans all of the supported resources within your zone of trust. When Access Analyzer finds a policy that allows access to a resource from outside of your zone of trust, it generates an active finding.
Incorrect options:
Access Advisor feature on IAM console - To help identify the unused roles, IAM reports the last-used timestamp that represents when a role was last used to make an AWS request. Your security team can use this information to identify, analyze, and then confidently remove unused roles. This helps improve the security posture of your AWS environments. This does not provide information about non-IAM entities such as S3, hence it's not a correct choice here.
S3 Object Lock - S3 Object Lock enables you to store objects using a "Write Once Read Many" (WORM) model. S3 Object Lock can help prevent accidental or inappropriate deletion of data, it is not the right choice for the current scenario.
S3 Analytics - By using Amazon S3 analytics Storage Class Analysis you can analyze storage access patterns to help you decide when to transition the right data to the right storage class. You cannot use S3 Analytics to identify unintended access to your S3 resources.

Question 9

The development team at a company creates serverless solutions using AWS Lambda. Functions are invoked by clients via AWS API Gateway which anyone can access. The team lead would like to control access using a 3rd party authorization mechanism.
As a Developer Associate, which of the following options would you recommend for the given use-case?

- IAM permissions with sigv4
- Cognito User Pools
- API Gateway User Pools
- Lambda Authorizer

Overall explanation
Correct option:
"Lambda Authorizer"
An Amazon API Gateway Lambda authorizer (formerly known as a custom authorizer) is a Lambda function that you provide to control access to your API. A Lambda authorizer uses bearer token authentication strategies, such as OAuth or SAML. Before creating an API Gateway Lambda authorizer, you must first create the AWS Lambda function that implements the logic to authorize and, if necessary, to authenticate the caller.



Use API Gateway Lambda authorizers

PDF | Kindle | RSS

A *Lambda authorizer* (formerly known as a *custom authorizer*) is an API Gateway feature that uses a Lambda function to control access to your API.

A Lambda authorizer is useful if you want to implement a custom authorization scheme that uses a bearer token authentication strategy such as OAuth or SAML, or that uses request parameters to determine the caller's identity.

When a client makes a request to one of your API's methods, API Gateway calls your Lambda authorizer, which takes the caller's identity as input and returns an IAM policy as output.

There are two types of Lambda authorizers:

- A *token-based* Lambda authorizer (also called a TOKEN authorizer) receives the caller's identity in a bearer token, such as a JSON Web Token (JWT) or an OAuth token.
- A *request parameter-based* Lambda authorizer (also called a REQUEST authorizer) receives the caller's identity in a combination of headers, query string parameters, stageVariables, and $context variables.

  For WebSocket APIs, only request parameter-based authorizers are supported.

It is possible to use an AWS Lambda function from an AWS account that is different from the one in which you created your Lambda authorizer function. For more information, see Configure a cross-account Lambda authorizer.

via - https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html
Incorrect options:
"IAM permissions with sigv4" - Signature Version 4 is the process to add authentication information to AWS requests sent by HTTP. You will still need to provide permissions but our requirements have a need for 3rd party authentication which is where Lambda Authorizer comes in to play.
"Cognito User Pools" - A Cognito user pool is a user directory in Amazon Cognito. With a user pool, your users can sign in to your web or mobile app through Amazon Cognito, or federate through a third-party identity provider (IdP). Whether your users sign-in directly or through a third party, all members of the user pool have a directory profile that you can access through an SDK. This is managed by AWS, therefore, does not meet our requirements.
"API Gateway User Pools" - This is a made-up option, added as a distractor.

Question 10

A developer has an application that stores data in an Amazon S3 bucket. The application uses an HTTP API to store and retrieve objects. When the PutObject API operation adds objects to the S3 bucket the developer must encrypt these objects at rest by using server-side encryption with Amazon S3-managed keys (SSE-S3).

Which solution will guarantee that any upload request without the mandated encryption is not processed?

- Invoke the PutObject API operation and set the x-amz-server-side-encryption header as aws:kms. Use an S3 bucket policy to deny permission to upload an object unless the request has this header
- Invoke the PutObject API operation and set the x-amz-server-side-encryption header as sse:s3. Use an S3 bucket policy to deny permission to upload an object unless the request has this header
- Invoke the PutObject API operation and set the x-amz-server-side-encryption header as AES256. Use an S3 bucket policy to deny permission to upload an object unless the request has this header
- Set the encryption key for SSE-S3 in the HTTP header of every request. Use an S3 bucket policy to deny permission to upload an object unless the request has this header

Overall explanation
Correct option:
Invoke the PutObject API operation and set the x-amz-server-side-encryption header as AES256. Use an S3 bucket policy to deny permission to upload an object unless the request has this header
SSE-S3 server-side encryption protects data at rest. Amazon S3 encrypts each object with a unique key. As an additional safeguard, it encrypts the key itself with a key that it rotates regularly. Amazon S3 server-side encryption uses one of the strongest block ciphers available to encrypt your data, 256-bit Advanced Encryption Standard (AES-256).
You can use the following bucket policy to deny permissions to upload an object unless the request includes the x-amz-server-side-encryption header to request server-side encryption using SSE-S3:

```
{
  "Version": "2012-10-17",
  "Id": "PutObjectPolicy",
  "Statement": [
    {
      "Sid": "DenyIncorrectEncryptionHeader",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::DOC-EXAMPLE-BUCKET/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "AES256"
        }
      }
    },
    {
      "Sid": "DenyUnencryptedObjectUploads",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::DOC-EXAMPLE-BUCKET/*",
      "Condition": {
        "Null": {
          "s3:x-amz-server-side-encryption": "true"
        }
      }
    }
  ]
}
```

Incorrect options:
Invoke the PutObject API operation and set the x-amz-server-side-encryption header as aws:kms. Use an S3 bucket policy to deny permission to upload an object unless the request has this header - As mentioned above, you need to use AES256 rather than aws:kms for the given use case. aws:kms is used when you want to use server-side encryption with AWS KMS (SSE-KMS).
Invoke the PutObject API operation and set the x-amz-server-side-encryption header as sse:s3. Use an S3 bucket policy to deny permission to upload an object unless the request has this header - This is a made-up option as the x-amz-server-side-encryption header has no such value as sse:s3.
Set the encryption key for SSE-S3 in the HTTP header of every request. Use an S3 bucket policy to deny permission to upload an object unless the request has this header - This option has been added as a distractor. For SSE-S3, Amazon S3 encrypts each object with a unique key. As an additional safeguard, it encrypts the key itself with a key that it rotates regularly. The encryption key for SSE-S3 encryption key cannot be accessed.

Question 11

A developer is defining the signers that can create signed URLs for their Amazon CloudFront distributions.
Which of the following statements should the developer consider while defining the signers? (Select two)

- When you create a signer, the public key is with CloudFront and private key is used to sign a portion of URL
- Both the signers (trusted key groups and CloudFront key pairs) can be managed using the CloudFront APIs
- You can also use AWS Identity and Access Management (IAM) permissions policies to restrict what the root user can do with CloudFront key pairs
- When you use the root user to manage CloudFront key pairs, you can only have up to two active CloudFront key pairs per AWS account
- CloudFront key pairs can be created with any account that has administrative permissions and full access to CloudFront resources

Overall explanation
Correct options:
When you create a signer, the public key is with CloudFront and private key is used to sign a portion of URL - Each signer that you use to create CloudFront signed URLs or signed cookies must have a public–private key pair. The signer uses its private key to sign the URL or cookies, and CloudFront uses the public key to verify the signature.
When you create signed URLs or signed cookies, you use the private key from the signer's key pair to sign a portion of the URL or the cookie. When someone requests a restricted file, CloudFront compares the signature in the URL or cookie with the unsigned URL or cookie, to verify that it hasn't been tampered with. CloudFront also verifies that the URL or cookie is valid, meaning, for example, that the expiration date and time haven't passed.
When you use the root user to manage CloudFront key pairs, you can only have up to two active CloudFront key pairs per AWS account - When you use the root user to manage CloudFront key pairs, you can only have up to two active CloudFront key pairs per AWS account.
Whereas, with CloudFront key groups, you can associate a higher number of public keys with your CloudFront distribution, giving you more flexibility in how you use and manage the public keys. By default, you can associate up to four key groups with a single distribution, and you can have up to five public keys in a key group.
Incorrect options:
You can also use AWS Identity and Access Management (IAM) permissions policies to restrict what the root user can do with CloudFront key pairs - When you use the AWS account root user to manage CloudFront key pairs, you can't restrict what the root user can do or the conditions in which it can do them. You can't apply IAM permissions policies to the root user, which is one reason why AWS best practices recommend against using the root user.
CloudFront key pairs can be created with any account that has administrative permissions and full access to CloudFront resources - CloudFront key pairs can only be created using the root user account and hence is not a best practice to create CloudFront key pairs as signers.
Both the signers (trusted key groups and CloudFront key pairs) can be managed using the CloudFront APIs - With CloudFront key groups, you can manage public keys, key groups, and trusted signers using the CloudFront API. You can use the API to automate key creation and key rotation. When you use the AWS root user, you have to use the AWS Management Console to manage CloudFront key pairs, so you can't automate the process.

Question 12

A company runs its flagship application on a fleet of Amazon EC2 instances. After misplacing a couple of private keys from the SSH key pairs, they have decided to re-use their SSH key pairs for the different instances across AWS Regions.
As a Developer Associate, which of the following would you recommend to address this use-case?

- Encrypt the private SSH key and store it in the S3 bucket to be accessed from any AWS Region
- It is not possible to reuse SSH key pairs across AWS Regions
- Store the public and private SSH key pair in AWS Trusted Advisor and access it across AWS Regions
- Generate a public SSH key from a private SSH key. Then, import the key into each of your AWS Regions

Overall explanation
Correct option:
Generate a public SSH key from a private SSH key. Then, import the key into each of your AWS Regions
Here is the correct way of reusing SSH keys in your AWS Regions:

1. Generate a public SSH key (.pub) file from the private SSH key (.pem) file.
2. Set the AWS Region you wish to import to.
3. Import the public SSH key into the new Region.

Incorrect options:
It is not possible to reuse SSH key pairs across AWS Regions - As explained above, it is possible to reuse with manual import.
Store the public and private SSH key pair in AWS Trusted Advisor and access it across AWS Regions - AWS Trusted Advisor is an application that draws upon best practices learned from

AWS' aggregated operational history of serving hundreds of thousands of AWS customers. Trusted Advisor inspects your AWS environment and makes recommendations for saving money, improving system performance, or closing security gaps. It does not store key pair credentials.
Encrypt the private SSH key and store it in the S3 bucket to be accessed from any AWS Region - Storing private key to Amazon S3 is possible. But, this will not make the key accessible for all AWS Regions, as is the need in the current use case.

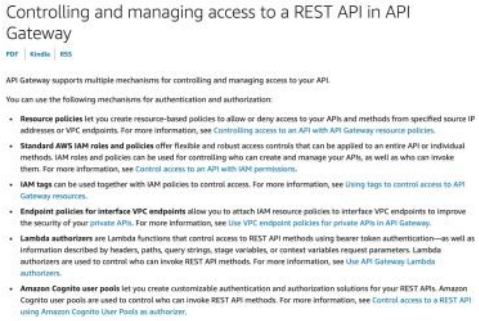
Question 13

A developer is looking at establishing access control for an API that connects to a Lambda function downstream.
Which of the following represents a mechanism that CANNOT be used for authenticating with the API Gateway?

- Lambda Authorizer
- AWS Security Token Service (STS)
- Cognito User Pools
- Standard AWS IAM roles and policies

Overall explanation
Correct option:
Amazon API Gateway is an AWS service for creating, publishing, maintaining, monitoring, and securing REST, HTTP, and WebSocket APIs at any scale. API developers can create APIs that access AWS or other web services, as well as data stored in the AWS Cloud.
How API Gateway Works:

via - https://aws.amazon.com/api-gateway/
AWS Security Token Service (STS) - AWS Security Token Service (AWS STS) is a web service that enables you to request temporary, limited-privilege credentials for AWS Identity and Access Management (IAM) users or for users that you authenticate (federated users). However, it is not supported by API Gateway.
API Gateway supports the following mechanisms for authentication and authorization:



Controlling and managing access to a REST API in API Gateway

PDF | Kindle | RSS

API Gateway supports multiple mechanisms for controlling and managing access to your API.

You can use the following mechanisms for authentication and authorization:

- **Resource policies** let you create resource-based policies to allow or deny access to your APIs and methods from specified source IP addresses or VPC endpoints. For more information, see Controlling access to an API with API Gateway resource policies.
- **Standard AWS IAM roles and policies** offer flexible and robust access controls that can be applied to an entire API or individual methods. IAM roles and policies can be used for controlling who can create and manage your APIs, as well as who can invoke them. For more information, see Control access to an API with IAM permissions.
- **IAM tags** can be used together with IAM policies to control access. For more information, see Using tags to control access to API Gateway resources.
- **Endpoint policies for interface VPC endpoints** allow you to attach IAM resource policies to interface VPC endpoints to improve the security of your private APIs. For more information, see Use VPC endpoint policies for private APIs in API Gateway.
- **Lambda authorizers** are Lambda functions that control access to REST API methods using bearer token authentication—as well as information described by headers, paths, query strings, stage variables, or context variables request parameters. Lambda authorizers are used to control who can invoke REST API methods. For more information, see Use API Gateway Lambda authorizers.
- **Amazon Cognito user pools** let you create customizable authentication and authorization solutions for your REST APIs. Amazon Cognito user pools are used to control who can invoke REST API methods. For more information, see Control access to a REST API using Amazon Cognito User Pools as authorizer.

via - https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-to-api.html
Incorrect options:
Standard AWS IAM roles and policies - Standard AWS IAM roles and policies offer flexible and robust access controls that can be applied to an entire API or individual methods. IAM roles and policies can be used for controlling who can create and manage your APIs, as well as who can invoke them.
Lambda Authorizer - Lambda authorizers are Lambda functions that control access to REST API methods using bearer token authentication—as well as information described by headers, paths, query strings, stage variables, or context variables request parameters. Lambda authorizers are used to control who can invoke REST API methods.
Cognito User Pools - Amazon Cognito user pools let you create customizable authentication and authorization solutions for your REST APIs. Amazon Cognito user pools are used to control who can invoke REST API methods.

Question 14

You have launched several AWS Lambda functions written in Java. A new requirement was given that over 1MB of data should be passed to the functions and should be encrypted and decrypted at runtime.
Which of the following methods is suitable to address the given use-case?

- Use KMS Encryption and store as environment variable
- Use KMS direct encryption and store as file
- Use Envelope Encryption and reference the data as file within the code
- Use Envelope Encryption and store as environment variable

Overall explanation
Correct option:
Use Envelope Encryption and reference the data as file within the code
While AWS KMS does support sending data up to 4 KB to be encrypted directly, envelope encryption can offer significant performance benefits. When you encrypt data directly with AWS KMS it must be transferred over the network. Envelope encryption reduces the network load since only the request and delivery of the much smaller data key go over the network. The data key is used locally in your application or encrypting AWS service, avoiding the need to send the entire block of data to AWS KMS and suffer network latency.
AWS Lambda environment variables can have a maximum size of 4 KB. Additionally, the direct 'Encrypt' API of KMS also has an upper limit of 4 KB for the data payload. To encrypt 1 MB, you need to use the Encryption SDK and pack the encrypted file with the lambda function.
Incorrect options:
Use KMS direct encryption and store as file - You can only encrypt up to 4 kilobytes (4096 bytes) of arbitrary data such as an RSA key, a database password, or other sensitive information, so this option is not correct for the given use-case.
Use Envelope Encryption and store as an environment variable - Environment variables must not exceed 4 KB, so this option is not correct for the given use-case.
Use KMS Encryption and store as an environment variable - You can encrypt up to 4 kilobytes (4096 bytes) of arbitrary data such as an RSA key, a database password, or other sensitive information. Lambda Environment variables must not exceed 4 KB. So this option is not correct for the given use-case.

Question 15

Consider an application that enables users to store their mobile phone images in the cloud and supports tens of thousands of users. The application should utilize an Amazon API Gateway REST API that leverages AWS Lambda functions for photo processing while storing photo details in Amazon DynamoDB. The application should allow users to create an account, upload images, and retrieve previously uploaded images, with images ranging in size from 500 KB to 5 MB.
How will you design the application with the least operational overhead?

- Use Cognito identity pools to manage user accounts and set up an Amazon Cognito identity pool authorizer in API Gateway to control access to the API. Set up a Lambda function to store the images in Amazon S3 and save the image object's S3 key as part of the photo details in a DynamoDB table. Have the Lambda function retrieve previously uploaded images by querying DynamoDB for the S3 key
- Use Cognito identity pools to create an IAM user for each user of the application during the sign-up process. Leverage IAM authentication in API Gateway to control access to the API. Set up a Lambda function to store the images in Amazon S3 and save the image object's S3 key as part of the photo details in a DynamoDB table. Have the Lambda function retrieve previously uploaded images by querying DynamoDB for the S3 key
- Leverage Cognito user pools to manage user accounts and set up an Amazon Cognito user pool authorizer in API Gateway to control access to the API. Set up a Lambda function to store the images in Amazon S3 and save the image object's S3 key as part of the photo details in a DynamoDB table. Have the Lambda function retrieve previously uploaded images by querying DynamoDB for the S3 key
- Leverage Cognito user pools to manage user accounts and set up an Amazon Cognito user pool authorizer in API Gateway to control access to the API. Set up a Lambda function to store the images as well as the image metadata in a DynamoDB table. Have the Lambda function retrieve previously uploaded images from DynamoDB

Overall explanation
Correct option:
Leverage Cognito user pools to manage user accounts and set up an Amazon Cognito user pool authorizer in API Gateway to control access to the API. Set up a Lambda function to store the images in Amazon S3 and save the image object's S3 key as part of the photo details in a DynamoDB table. Have the Lambda function retrieve previously uploaded images by querying DynamoDB for the S3 key
A user pool is a user directory in Amazon Cognito. With a user pool, your users can sign in to your web or mobile app through Amazon Cognito. Your users can also sign in through social identity providers like Google, Facebook, Amazon, or Apple, and SAML identity providers. Whether your users sign in directly or through a third party, all members of the user pool have a directory profile that you can access through a Software Development Kit (SDK).
User pools provide:
Sign-up and sign-in services.
A built-in, customizable web UI to sign in users.
Social sign-in with Facebook, Google, Login with Amazon, and Sign in with Apple, as well as sign-in with SAML identity providers from your user pool.
User directory management and user profiles.
Security features such as multi-factor authentication (MFA), checks for compromised credentials, account takeover protection, and phone and email verification.
Customized workflows and user migration through AWS Lambda triggers.
To use an Amazon Cognito user pool with your Amazon API Gateway API, you must first create an authorizer of the COGNITO_USER_POOLS type and then configure an API method to use that authorizer. After the API is deployed, the client must first sign the user into the user pool, obtain an identity or access token for the user, and then call the API method with one of the tokens, which are typically set to the request's Authorization header.
For the given use case, you can use a Cognito user pool to manage user accounts and configure an Amazon Cognito user pool authorizer in API Gateway to control access to the API. You should use a Lambda function to store the actual images on S3 and the image metadata on DynamoDB. Finally, you can get the images using the Lambda function that leverages the metadata stored in DynamoDB.
Incorrect options:
Use Cognito identity pools to manage user accounts and set up an Amazon Cognito identity pool authorizer in API Gateway to control access to the API. Set up a Lambda function to store the images in Amazon S3 and save the image object's S3 key as part of the photo details in a DynamoDB table. Have the Lambda function retrieve previously uploaded images by querying DynamoDB for the S3 key
Use Cognito identity pools to create an IAM user for each user of the application during the sign-up process. Leverage IAM authentication in API Gateway to control access to the API. Set up a Lambda function to store the images in Amazon S3 and save the image object's S3 key as part of the photo details in a DynamoDB table. Have the Lambda function retrieve previously uploaded images by querying DynamoDB for the S3 key
Amazon Cognito identity pools (federated identities) enable you to create unique identities for your users and federate them with identity providers. With an identity pool, you can obtain temporary, limited-privilege AWS credentials to access other AWS services. You cannot use identity pools to manage users or to create IAM users. So both of these options are incorrect.


What's the difference between Amazon Cognito user pools and identity pools?

Last updated: 2021-03-05

I'm starting to use Amazon Cognito, and I'm not sure whether I should use user pools or identity pools for my business applications. What's the difference?

Short description

User pools are for authentication (identity verification). With a user pool, your app users can sign in through the user pool or federate through a third-party identity provider (IdP).

Identity pools are for authorization (access control). You can use identity pools to create unique identities for users and give them access to other AWS services.

Resolution

User pool use cases

Use a user pool when you need to:

- Design sign-up and sign-in webpages for your app.
- Access and manage user data.
- Track user device, location, and IP address, and adapt to sign-in requests of different risk levels.
- Use a custom authentication flow for your app.

Identity pool use cases

Use an identity pool when you need to:

- Give your users access to AWS resources, such as an Amazon Simple Storage Service (Amazon S3) bucket or an Amazon DynamoDB table.
- Generate temporary AWS credentials for unauthenticated users.

via - https://aws.amazon.com/premiumsupport/knowledge-center/cognito-user-pools-identity-pools/
Leverage Cognito user pools to manage user accounts and set up an Amazon Cognito user pool authorizer in API Gateway to control access to the API. Set up a Lambda function to store the images as well as the image metadata in a DynamoDB table. Have the Lambda function retrieve previously uploaded images from DynamoDB - You cannot use DynamoDB to store images as the maximum allowed item size is 400KB and the images range in size from 500KB to 5MB. You should also note that storing images on DynamoDB is an anti-pattern. So this option is incorrect.

Question 16

A large firm stores its static data assets on Amazon S3 buckets. Each service line of the firm has its own AWS account. For a business use case, the Finance department needs to give access to their S3 bucket's data to the Human Resources department.
Which of the below options is NOT feasible for cross-account access of S3 bucket objects?

Use Access Control List (ACL) and IAM policies for programmatic-only access to S3 bucket objects
Use Cross-account IAM roles for programmatic and console access to S3 bucket objects
Use IAM roles and resource-based policies delegate access across accounts within different partitions via programmatic access only
Use Resource-based policies and AWS Identity and Access Management (IAM) policies for programmatic-only access to S3 bucket objects

Overall explanation
Correct option:
Use
IAM roles and resource-based policies delegate access across accounts within different partitions via programmatic access only - This statement is incorrect and hence the right choice for this question. IAM roles and resource-based policies delegate access across accounts only within a single partition. For example, assume that you have an account in US West (N. California) in the standard aws partition. You also have an account in China (Beijing) in the aws-cn partition. You can't use an Amazon S3 resource-based policy in your account in China (Beijing) to allow access for users in your standard AWS account.
Incorrect options:
Use Resource-based policies and AWS Identity and Access Management (IAM) policies for programmatic-only access to S3 bucket objects - Use bucket policies to manage cross-account control and audit the S3 object's permissions. If you apply a bucket policy at the bucket level, you can define who can access (Principal element), which objects they can access (Resource element), and how they can access (Action element). Applying a bucket policy at the bucket level allows you to define granular access to different objects inside the bucket by using multiple policies to control access. You can also review the bucket policy to see who can access objects in an S3 bucket.
Use Access Control List (ACL) and IAM policies for programmatic-only access to S3 bucket objects - Use object ACLs to manage permissions only for specific scenarios and only if ACLs meet your needs better than IAM and S3 bucket policies. Amazon S3 ACLs allow users to define only the following permissions sets: READ, WRITE, READ_ACP, WRITE_ACP, and FULL_CONTROL. You can use only an AWS account or one of the predefined Amazon S3 groups as a grantee for the Amazon S3 ACL.
Use Cross-account IAM roles for programmatic and console access to S3 bucket objects - Not all AWS services support resource-based policies. This means that you can use cross-account IAM roles to centralize permission management when providing cross-account access to multiple services. Using cross-account IAM roles simplifies provisioning cross-account access to S3 objects that are stored in multiple S3 buckets, removing the need to manage multiple policies for S3 buckets. This method allows cross-account access to objects that are owned or uploaded by another AWS account or AWS services. If you don't use cross-account IAM roles, the object ACL must be modified.

Question 17

A telecom service provider stores its critical customer data on Amazon Simple Storage Service (Amazon S3).
Which of the following options can be used to control access to data stored on Amazon S3? (Select two)

IAM database authentication, Bucket policies
Permissions boundaries, Identity and Access Management (IAM) policies
Query String Authentication, Permissions boundaries
Bucket policies, Identity and Access Management (IAM) policies
Query String Authentication, Access Control Lists (ACLs)

Overall explanation
Correct options:
Bucket policies, Identity and Access Management (IAM) policies
Query String Authentication, Access Control Lists (ACLs)
Customers may use four mechanisms for controlling access to Amazon S3 resources: Identity and Access Management (IAM) policies, bucket policies, Access Control Lists (ACLs), and Query String Authentication.
IAM enables organizations with multiple employees to create and manage multiple users under a single AWS account. With IAM policies, customers can grant IAM users fine-grained control to their Amazon S3 bucket or objects while also retaining full control over everything the users do.
With bucket policies, customers can define rules which apply broadly across all requests to their Amazon S3 resources, such as granting write privileges to a subset of Amazon S3 resources. Customers can also restrict access based on an aspect of the request, such as HTTP referrer and IP address.
With ACLs, customers can grant specific permissions (i.e. READ, WRITE, FULL_CONTROL) to specific users for an individual bucket or object.

With Query String Authentication, customers can create a URL to an Amazon S3 object which is only valid for a limited time. Using query parameters to authenticate requests is useful when you want to express a request entirely in a URL. This method is also referred as presigning a URL.
Incorrect options:
Permissions boundaries, Identity and Access Management (IAM) policies
Query String Authentication, Permissions boundaries
IAM database authentication, Bucket policies
Permissions boundary - A Permissions boundary is an advanced feature for using a managed policy to set the maximum permissions that an identity-based policy can grant to an IAM entity. An entity's permissions boundary allows it to perform only the actions that are allowed by both its identity-based policies and its permissions boundaries. When you use a policy to set the permissions boundary for a user, it limits the user's permissions but does not provide permissions on its own.
IAM database authentication - IAM database authentication works with MySQL and PostgreSQL. With this authentication method, you don't need to use a password when you connect to a DB instance. Instead, you use an authentication token. It is a database authentication technique and cannot be used to authenticate for S3.
Therefore, all three options are incorrect.

Question 18

The Development team at a media company is working on securing their databases.
Which of the following AWS database engines can be configured with IAM Database Authentication? (Select two)

RDS SQL Server
RDS MySQL
RDS Oracle
RDS Db2
RDS PostGreSQL
Overall explanation

Correct options:
You can authenticate to your DB instance using AWS Identity and Access Management (IAM) database authentication. With this authentication method, you don't need to use a password when you connect to a DB instance. Instead, you use an authentication token. An authentication token is a unique string of characters that Amazon RDS generates on request. Each token has a lifetime of 15 minutes. You don't need to store user credentials in the database, because authentication is managed externally using IAM.
RDS MySQL - IAM database authentication works with MySQL and PostgreSQL engines for Aurora as well as MySQL, MariaDB and RDS PostgreSQL engines for RDS.
RDS PostGreSQL - IAM database authentication works with MySQL and PostgreSQL engines for Aurora as well as MySQL, MariaDB and RDS PostgreSQL engines for RDS.
Incorrect options:
RDS Oracle
RDS SQL Server
These two options contradict the details in the explanation above, so these are incorrect.
RDS Db2 - This option has been added as a distractor. Db2 is a family of data management products, including database servers, developed by IBM. RDS does not support Db2 database engine.

Question 19

A developer wants to integrate user-specific file upload and download features in an application that uses both Amazon Cognito user pools and Cognito identity pools for secure access with Amazon S3. The developer also wants to ensure that only authorized users can access their own files and that the files are securely saved and retrieved. The files are 5 KB to 500 MB in size.
What do you recommend as the most efficient solution?
Correct answer

- Leverage an IAM policy with the Amazon Cognito identity prefix to allow users access to just their own objects in Amazon S3
- Integrate Amazon API Gateway with a Lambda function that validates that the given file is uploaded to S3 and downloaded from S3 only by the authorized user
- Use CloudFront Lambda@Edge to validate that the given file is uploaded to S3 and downloaded from S3 only by the authorized user
- Use S3 Event Notifications to trigger a Lambda function that validates that the given file is uploaded and downloaded only by the authorized user

Overall explanation
Correct option:
Leverage an IAM policy with the Amazon Cognito identity prefix to allow users access to just their own objects in Amazon S3
Amazon Cognito identity pools (federated identities) enable you to create unique identities for your users and federate them with identity providers. With an identity pool, you can obtain temporary, limited-privilege AWS credentials to access other AWS services. Amazon Cognito identity pools support the following identity providers:
Public providers: Login with Amazon (identity pools), Facebook (identity pools), Google (identity pools), Sign in with Apple (identity pools).
Amazon Cognito user pools
OpenID Connect providers (identity pools)
SAML identity providers (identity pools)
Developer authenticated identities (identity pools)
You can create an identity-based policy that allows Amazon Cognito users to access objects in a specific S3 bucket. This policy allows access only to objects with a name that includes Cognito, the name of the application, and the federated user's ID, represented by the ${cognito-identity.amazonaws.com:sub} variable.

via - https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_examples_s3_cognito-bucket.html
Incorrect options:
Use S3 Event Notifications to trigger a Lambda function that validates that the given file is uploaded and downloaded only by the authorized user - While it is certainly possible to build this solution, however, it is not the most optimal solution as it does not prevent an invalid upload of a file into another user's designated folder. So this option is incorrect.
Integrate Amazon API Gateway with a Lambda function that validates that the given file is uploaded to S3 and downloaded from S3 only by the authorized user - Again, it is certainly possible to build this solution, however, it is not the most optimal solution as it does not prevent an invalid upload of a file into another user's designated folder. So this option is incorrect.
Use CloudFront Lambda@Edge to validate that the given file is uploaded to S3 and downloaded from S3 only by the authorized user - This option assumes that the solution comprises a CloudFront distribution. This introduces inefficiency in the solution, as one needs to pay for CloudFront/Lambda@Edge and adds unnecessary hops in the data flow for both uploads and downloads.

Question 20

You team maintains a public API Gateway that is accessed by clients from another domain. Usage has been consistent for the last few months but recently it has more than doubled. As a result, your costs have gone up and would like to prevent other unauthorized domains from accessing your API.
Which of the following actions should you take?

Assign a Security Group to your API Gateway
Restrict access by using CORS
Use Account-level throttling
Use Mapping Templates

Overall explanation
Correct option:
Restrict access by using CORS - Cross-origin resource sharing (CORS) defines a way for client web applications that are loaded in one domain to interact with resources in a different domain. When your API's resources receive requests from a domain other than the API's own domain and you want to restrict servicing these requests, you must disable cross-origin resource sharing (CORS) for selected methods on the resource.
Incorrect options:
Use Account-level throttling - To prevent your API from being overwhelmed by too many requests, Amazon API Gateway throttles requests to your API. By default, API Gateway limits the steady-state request rate to 10,000 requests per second (rps). It limits the burst (that is, the maximum bucket size) to 5,000 requests across all APIs within an AWS account. This is Account-level throttling. As

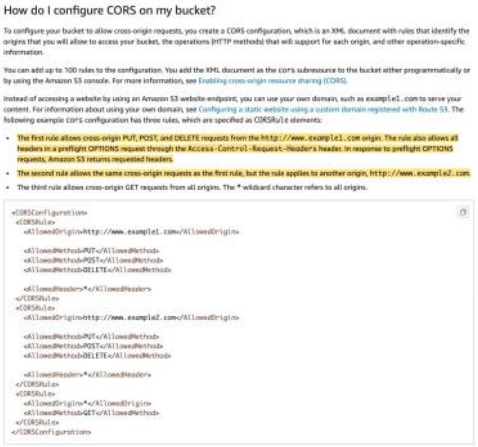
you see, this is about limit on the number of requests and is not a suitable answer for the current scenario.
Use Mapping Templates - A mapping template is a script expressed in Velocity Template Language (VTL) and applied to the payload using JSONPath expressions. Mapping templates help format/structure the data in a way that it is easily readable, unlike a server response that might always be easy to ready. Mapping Templates have nothing to do with access and are not useful for the current scenario.
Assign a Security Group to your API Gateway - API Gateway does not use security groups but uses resource policies, which are JSON policy documents that you attach to an API to control whether a specified principal (typically an IAM user or role) can invoke the API. You can restrict IP address using this, the downside being, an IP address can be changed by the accessing user. So, this is not an optimal solution for the current use case.

Question 21

A digital marketing company has its website hosted on an Amazon S3 bucket A. The development team notices that the web fonts that are hosted on another S3 bucket B are not loading on the website.
Which of the following solutions can be used to address this issue?

- Enable versioning on both the buckets to facilitate correct functioning of the website
- Configure CORS on the bucket A that is hosting the website to allow any origin to respond to requests
- Configure CORS on the bucket B that is hosting the web fonts to allow Bucket A origin to make the requests
- Update bucket policies on both bucket A and bucket B to allow successful loading of the web fonts on the website

Overall explanation
Correct option:
Configure CORS on the bucket B that is hosting the web fonts to allow Bucket A origin to make the requests
Cross-origin resource sharing (CORS) defines a way for client web applications that are loaded in one domain to interact with resources in a different domain.
To configure your bucket to allow cross-origin requests, you create a CORS configuration, which is an XML document with rules that identify the origins that you will allow to access your bucket, the operations (HTTP methods) that will support for each origin, and other operation-specific information.
For the given use-case, you would create a <CORSRule> in <CORSConfiguration> for bucket B to allow access from the S3 website origin hosted on bucket A.
Please see this note for more details:


How do I configure CORS on my bucket?

To configure your bucket to allow cross-origin requests, you create a CORS configuration, which is an XML document with rules that identify the origins that you will allow to access your bucket, the operations (HTTP methods) that will support for each origin, and other operation-specific information.

You can add up to 100 rules to the configuration. You add the XML document as the cors subresource to the bucket either programmatically or by using the Amazon S3 console. For more information, see Enabling cross-origin resource sharing (CORS).

Instead of accessing a website by using an Amazon S3 website endpoint, you can use your own domain, such as example1.com to serve your content. For information about using your own domain, see Configuring a static website using a custom domain registered with Route 53. The following example cors configuration has three rules, which are specified as CORSRule elements:

- The first rule allows cross-origin PUT, POST, and DELETE requests from the http://www.example1.com origin. The rule also allows all headers in a preflight OPTIONS request through the Access-Control-Request-Headers header. In response to preflight OPTIONS requests, Amazon S3 returns requested headers.
- The second rule allows the same cross-origin requests as the first rule, but the rule applies to another origin, http://www.example2.com.
- The third rule allows cross-origin GET requests from all origins. The * wildcard character refers to all origins.

```
<CORSConfiguration>
 <CORSRule>
   <AllowedOrigin>http://www.example1.com</AllowedOrigin>

   <AllowedMethod>PUT</AllowedMethod>
   <AllowedMethod>POST</AllowedMethod>
   <AllowedMethod>DELETE</AllowedMethod>

   <AllowedHeader>*</AllowedHeader>
 </CORSRule>
 <CORSRule>
   <AllowedOrigin>http://www.example2.com</AllowedOrigin>

   <AllowedMethod>PUT</AllowedMethod>
   <AllowedMethod>POST</AllowedMethod>
   <AllowedMethod>DELETE</AllowedMethod>

   <AllowedHeader>*</AllowedHeader>
 </CORSRule>
 <CORSRule>
   <AllowedOrigin>*</AllowedOrigin>
   <AllowedMethod>GET</AllowedMethod>
 </CORSRule>
</CORSConfiguration>
```

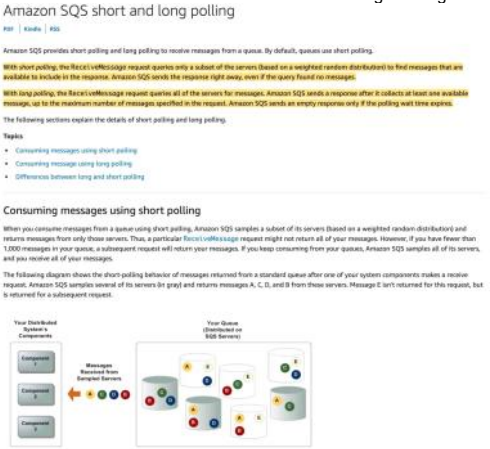
via - https://docs.aws.amazon.com/AmazonS3/latest/dev/cors.html
Incorrect options:
Enable versioning on both the buckets to facilitate the correct functioning of the website - This option is a distractor and versioning will not help to address the web fonts loading issue on the website.
Update bucket policies on both bucket A and bucket B to allow successful loading of the web fonts on the website - It's not the bucket policies but the CORS configuration on bucket B that needs to be updated to allow web fonts to be loaded on the website. Updating bucket policies will not help to address the web fonts loading issue on the website.
Configure CORS on the bucket A that is hosting the website to allow any origin to respond to requests - The CORS configuration needs to be updated on bucket B to allow web fonts to be loaded on the website hosted on bucket A. So this option in incorrect.

Question 22

A high-frequency stock trading firm is migrating their messaging queues from self-managed message-oriented middleware systems to Amazon SQS. The development team at the company wants to minimize the costs of using SQS.
As a Developer Associate, which of the following options would you recommend to address the given use-case?

- Use SQS message timer to retrieve messages from your Amazon SQS queues
- Use SQS long polling to retrieve messages from your Amazon SQS queues
- Use SQS visibility timeout to retrieve messages from your Amazon SQS queues
- Use SQS short polling to retrieve messages from your Amazon SQS queues

Overall explanation
Correct option:
Use SQS long polling to retrieve messages from your Amazon SQS queues
Amazon Simple Queue Service (SQS) is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications.
Amazon SQS provides short polling and long polling to receive messages from a queue. By default, queues use short polling. With short polling, Amazon SQS sends the response right away, even if the query found no messages. With long polling, Amazon SQS sends a response after it collects at least one available message, up to the maximum number of messages specified in the request. Amazon SQS sends an empty response only if the polling wait time expires.
Long polling makes it inexpensive to retrieve messages from your Amazon SQS queue as soon as the messages are available. Long polling helps reduce the cost of using Amazon SQS by eliminating the number of empty responses (when there are no messages available for a ReceiveMessage request) and false empty responses (when messages are available but aren't included in a response).
When the wait time for the ReceiveMessage API action is greater than 0, long polling is in effect. The maximum long polling wait time is 20 seconds.
Exam Alert:
Please review the differences between Short Polling vs Long Polling:


Amazon SQS short and long polling

PDF | Kindle | RSS

Amazon SQS provides short polling and long polling to receive messages from a queue. By default, queues use short polling.

With short polling, the ReceiveMessage request queries only a subset of the servers (based on a weighted random distribution) to find messages that are available to include in the response. Amazon SQS sends the response right away, even if the query found no messages.

With long polling, the ReceiveMessage request queries all of the servers for messages. Amazon SQS sends a response after it collects at least one available message, up to the maximum number of messages specified in the request. Amazon SQS sends an empty response only if the polling wait time expires.

The following sections explain the details of short polling and long polling.

Topics

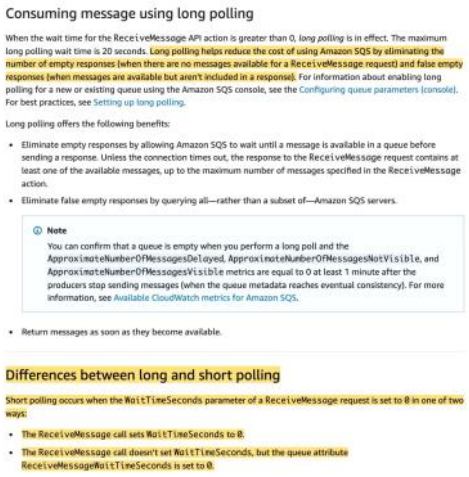
- Consuming messages using short polling
- Consuming message using long polling
- Differences between long and short polling

Consuming messages using short polling

When you consume messages from a queue using short polling, Amazon SQS samples a subset of its servers (based on a weighted random distribution) and returns messages from only those servers. Thus, a particular ReceiveMessage request might not return all of your messages. However, if you have fewer than 1,000 messages in your queue, a subsequent request will return your messages. If you keep consuming from your queues, Amazon SQS samples all of its servers, and you receive all of your messages.

The following diagram shows the short-polling behavior of messages returned from a standard queue after one of your system components makes a receive request. Amazon SQS samples several of its servers (in gray) and returns messages A, C, D, and B from those servers. Message E isn't returned for this request, but is returned for a subsequent request.

via - https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-short-and-long-polling.html



Consuming message using long polling

When the wait time for the ReceiveMessage API action is greater than 0, long polling is in effect. The maximum long polling wait time is 20 seconds. Long polling helps reduce the cost of using Amazon SQS by eliminating the number of empty responses (when there are no messages available for a ReceiveMessage request) and false empty responses (when messages are available but aren't included in a response). For information about enabling long polling for a new or existing queue using the Amazon SQS console, see the Configuring queue parameters (console). For best practices, see Setting up long polling.

Long polling offers the following benefits:

- Eliminate empty responses by allowing Amazon SQS to wait until a message is available in a queue before sending a response. Unless the connection times out, the response to the ReceiveMessage request contains at least one of the available messages, up to the maximum number of messages specified in the ReceiveMessage action.
- Eliminate false empty responses by querying all—rather than a subset of—Amazon SQS servers.

Note

You can confirm that a queue is empty when you perform a long poll and the ApproximateNumberOfMessagesDelayed, ApproximateNumberOfMessagesNotVisible, and ApproximateNumberOfMessagesVisible metrics are equal to 0 at least 1 minute after the producers stop sending messages (when the queue metadata reaches eventual consistency). For more information, see Available CloudWatch metrics for Amazon SQS.

- Return messages as soon as they become available.

Differences between long and short polling

Short polling occurs when the WaitTimeSeconds parameter of a ReceiveMessage request is set to 0 in one of two ways:

- The ReceiveMessage call sets WaitTimeSeconds to 0.
- The ReceiveMessage call doesn't set WaitTimeSeconds, but the queue attribute ReceiveMessageWaitTimeSeconds is set to 0.

via - https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-short-and-long-polling.html

Incorrect options:

Use SQS short polling to retrieve messages from your Amazon SQS queues - With short polling, Amazon SQS sends the response right away, even if the query found no messages. You end up paying more because of the increased number of empty receives.

Use SQS visibility timeout to retrieve messages from your Amazon SQS queues - Visibility timeout is a period during which Amazon SQS prevents other consumers from receiving and processing a given message. The default visibility timeout for a message is 30 seconds. The minimum is 0 seconds. The maximum is 12 hours. You cannot use visibility timeout to retrieve messages from your Amazon SQS queues. This option has been added as a distractor.

Use SQS message timer to retrieve messages from your Amazon SQS queues - You can use message timers to set an initial invisibility period for a message added to a queue. So, if you send a message with a 60-second timer, the message isn't visible to consumers for its first 60 seconds in the queue. The default (minimum) delay for a message is 0 seconds. The maximum is 15 minutes. You cannot use message timer to retrieve messages from your Amazon SQS queues. This option has been added as a distractor.

Question 23

A company developed an app-based service for citizens to book transportation rides in the local community. The platform is running on AWS EC2 instances and uses Amazon Relational Database Service (RDS) for storing transportation data. A new feature has been requested where receipts would be emailed to customers with PDF attachments retrieved from Amazon Simple Storage Service (S3).

Which of the following options will provide EC2 instances with the right permissions to upload files to Amazon S3 and generate S3 Signed URL?

- CloudFormation
- Create an IAM Role for EC2
- EC2 User Data
- Run aws configure on the EC2 instance

Overall explanation

Correct option:

Create an IAM Role for EC2

IAM roles have been incorporated so that your applications can securely make API requests from your instances, without requiring you to manage the security credentials that the applications use. Instead of creating and distributing your AWS credentials, you can delegate permission to make API requests using IAM roles.

Amazon EC2 uses an instance profile as a container for an IAM role. When you create an IAM role using the IAM console, the console creates an instance profile automatically and gives it the same name as the role to which it corresponds.

Incorrect options:

EC2 User Data - You can specify user data when you launch an instance and you would not want to hard code the AWS credentials in the user data.

Run aws configure on the EC2 instance - When you first configure the CLI you have to run this command, afterward you should not need to if you want to obtain credentials to authenticate to other AWS services. An IAM role will receive temporary credentials for you so you can focus on using the CLI to get access to other AWS services if you have the permissions.

CloudFormation - AWS CloudFormation gives developers and businesses an easy way to create a collection of related AWS and third-party resources and provision them in an orderly and predictable fashion.

How CloudFormation Works:

via - https://aws.amazon.com/cloudformation/

Question 24

An Amazon Simple Queue Service (SQS) has to be configured between two AWS accounts for shared access to the queue. AWS account A has the SQS queue in its account and AWS account B has to be given access to this queue.

Which of the following options need to be combined to allow this cross-account access? (Select three)

- The account B administrator creates an IAM role and attaches a trust policy to the role with account B as the principal
- The account B administrator delegates the permission to assume the role to any users in account B
- The account A administrator attaches a trust policy to the role that identifies account B as the principal who can assume the role
- The account A administrator creates an IAM role and attaches a permissions policy
- The account A administrator attaches a trust policy to the role that identifies account B as the AWS service principal who can assume the role
- The account A administrator delegates the permission to assume the role to any users in account A

Overall explanation

Correct options:

The account A administrator creates an IAM role and attaches a permissions policy

The account A administrator attaches a trust policy to the role that identifies account B as the principal who can assume the role

The account B administrator delegates the permission to assume the role to any users in account B

To grant cross-account permissions, you need to attach an identity-based permissions policy to an IAM role. For example, the AWS account A administrator can create a role to grant cross-account permissions to AWS account B as follows:

4. The account A administrator creates an IAM role and attaches a permissions policy—that grants permissions on resources in account A—to the role.
5. The account A administrator attaches a trust policy to the role that identifies account B as the principal who can assume the role.
6. The account B administrator delegates the permission to assume the role to any users in account B. This allows users in account B to create or access queues in account A.

Incorrect options:

The account B administrator creates an IAM role and attaches a trust policy to the role with account B as the principal - As mentioned above, the account A administrator needs to create an IAM role and then attach a permissions policy. So, this option is incorrect.

The account A administrator delegates the permission to assume the role to any users in account A - This is irrelevant, as users in account B need to be given access.

The account A administrator attaches a trust policy to the role that identifies account B as the AWS service principal who can assume the role - AWS service principal is given as principal in the trust policy when you need to grant the permission to assume the role to an AWS service. The given use case talks about giving permission to another account. So, service principal is not an option here.

Question 25

A cybersecurity company is publishing critical log data to a log group in Amazon CloudWatch Logs, which was created 3 months ago. The company must encrypt the log data using an AWS KMS customer master key (CMK), so any future data can be encrypted to meet the company's security guidelines.
How can the company address this use-case?

- Enable the encrypt feature on the log group via the CloudWatch Logs console
- Use the AWS CLI create-log-group command and specify the KMS key ARN
- Use the AWS CLI describe-log-groups command and specify the KMS key ARN
- Use the AWS CLI associate-kms-key command and specify the KMS key ARN

Overall explanation
Correct option:
Use the AWS CLI associate-kms-key command and specify the KMS key ARN
Log group data is always encrypted in CloudWatch Logs. You can optionally use AWS AWS Key Management Service for this encryption. If you do, the encryption is done using an AWS KMS (AWS KMS) customer master key (CMK). Encryption using AWS KMS is enabled at the log group level, by associating a CMK with a log group, either when you create the log group or after it exists.
After you associate a CMK with a log group, all newly ingested data for the log group is encrypted using the CMK. This data is stored in an encrypted format throughout its retention period.
CloudWatch Logs decrypts this data whenever it is requested. CloudWatch Logs must have permissions for the CMK whenever encrypted data is requested.
To associate the CMK with an existing log group, you can use the associate-kms-key command.

via - https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/encrypt-log-data-kms.html
Incorrect options:
Enable the encrypt feature on the log group via the CloudWatch Logs console - CloudWatch Logs console does not have an option to enable encryption for a log group.
Use the AWS CLI describe-log-groups command and specify the KMS key ARN - You can use the describe-log-groups command to find whether a log group already has a CMK associated with it.
Use the AWS CLI create-log-group command and specify the KMS key ARN - You can use the create-log-group command to associate the CMK with a log group when you create it.

Question 26

An organization recently began using AWS CodeCommit for its source control service. A compliance security team visiting the organization was auditing the software development process and noticed developers making many git push commands within their development machines. The compliance team requires that encryption be used for this activity.
How can the organization ensure source code is encrypted in transit and at rest?

- Enable KMS encryption
- Repositories are automatically encrypted at rest
- Use a git command line hook to encrypt the code client side
- Use AWS Lambda as a hook to encrypt the pushed code

Overall explanation
Correct option:
Repositories are automatically encrypted at rest
Data in AWS CodeCommit repositories is encrypted in transit and at rest. When data is pushed into an AWS CodeCommit repository (for example, by calling git push), AWS CodeCommit encrypts the received data as it is stored in the repository.

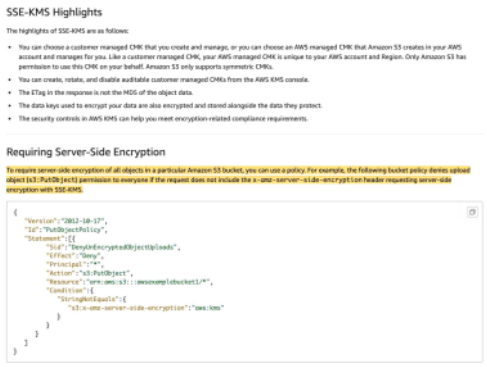
via - https://docs.aws.amazon.com/codecommit/latest/userguide/encryption.html
Incorrect options:
Enable KMS encryption - You don't have to. The first time you create an AWS CodeCommit repository in a new region in your AWS account, CodeCommit creates an AWS-managed key in that same region in AWS Key Management Service (AWS KMS) that is used only by CodeCommit.
Use AWS Lambda as a hook to encrypt the pushed code - This is not needed as CodeCommit handles it for you.
Use a git command line hook to encrypt the code client-side - This is not needed as CodeCommit handles it for you.

Question 27

Your development team uses the AWS SDK for Java on a web application that uploads files to several Amazon Simple Storage Service (S3) buckets using the SSE-KMS encryption mechanism. Developers are reporting that they are receiving permission errors when trying to push their objects over HTTP. Which of the following headers should they include in their request?

'x-amz-server-side-encryption': 'AES256'
'x-amz-server-side-encryption': 'SSE-S3'
'x-amz-server-side-encryption': 'SSE-KMS'
'x-amz-server-side-encryption': 'aws:kms'

Overall explanation
Correct option:
'x-amz-server-side-encryption': 'aws:kms'
Server-side encryption is the encryption of data at its destination by the application or service that receives it. AWS Key Management Service (AWS KMS) is a service that combines secure, highly available hardware and software to provide a key management system scaled for the cloud. Amazon S3 uses AWS KMS customer master keys (CMKs) to encrypt your Amazon S3 objects. AWS KMS encrypts only the object data. Any object metadata is not encrypted.
If the request does not include the x-amz-server-side-encryption header, then the request is denied.



via - https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingKMSEncryption.html

Incorrect options:

'x-amz-server-side-encryption': 'SSE-S3' - This is an invalid header value. The correct value is 'x-amz-server-side-encryption': 'AES256'. This refers to Server-Side Encryption with Amazon S3-Managed Encryption Keys (SSE-S3).

'x-amz-server-side-encryption': 'SSE-KMS' - Invalid header value. SSE-KMS is an encryption option.

'x-amz-server-side-encryption': 'AES256' - This is the correct header value if you are using SSE-S3 server-side encryption.

Question 28

You have a popular web application that accesses data stored in an Amazon Simple Storage Service (S3) bucket. Developers use the SDK to maintain the application and add new features. Security compliance requests that all new objects uploaded to S3 be encrypted using SSE-S3 at the time of upload. Which of the following headers must the developers add to their request?

- 'x-amz-server-side-encryption': 'SSE-KMS'
- 'x-amz-server-side-encryption': 'AES256'
- 'x-amz-server-side-encryption': 'SSE-S3'
- 'x-amz-server-side-encryption': 'aws:kms'

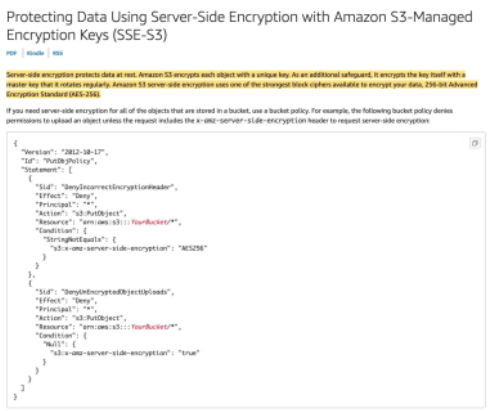
Overall explanation

Correct option:

'x-amz-server-side-encryption': 'AES256'

Server-side encryption protects data at rest. Amazon S3 encrypts each object with a unique key. As an additional safeguard, it encrypts the key itself with a master key that it rotates regularly. Amazon S3 server-side encryption uses one of the strongest block ciphers available to encrypt your data, 256-bit Advanced Encryption Standard (AES-256).

SSE-S3 Overview:



via - https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingServerSideEncryption.html

'x-amz-server-side-encryption': 'SSE-S3' - SSE-S3 (Amazon S3-Managed Keys) is an option available but it's not a valid header value.

'x-amz-server-side-encryption': 'SSE-KMS' - SSE-KMS (AWS KMS-Managed Keys) is an option available but it's not a valid header value. A valid value would be 'aws:kms'

'x-amz-server-side-encryption': 'aws:kms' - Server-side encryption is the encryption of data at its destination by the application or service that receives it. AWS Key Management Service (AWS KMS) is a service that combines secure, highly available hardware and software to provide a key management system scaled for the cloud. Amazon S3 uses AWS KMS customer master keys (CMKs) to encrypt your Amazon S3 objects. AWS KMS encrypts only the object data. Any object metadata is not encrypted.

This is a valid header value and you can use if you need more control over your keys like create, rotating, disabling them using AWS KMS. Otherwise, if you wish to let AWS S3 manage your keys just stick with SSE-S3.

Question 29

A development team is storing sensitive customer data in S3 that will require encryption at rest. The encryption keys must be rotated at least annually.

What is the easiest way to implement a solution for this requirement?

- Use AWS KMS with automatic key rotation
- Import a custom key into AWS KMS and automate the key rotation on an annual basis by using a Lambda function
- Encrypt the data before sending it to Amazon S3
- Use SSE-C with automatic key rotation on an annual basis

Overall explanation

Correct option:

Use AWS KMS with automatic key rotation - Server-side encryption is the encryption of data at its destination by the application or service that receives it. Amazon S3 encrypts your data at the object level as it writes it to disks in its data centers and decrypts it for you when you access it. You have three mutually exclusive options, depending on how you choose to manage the encryption keys: Server-Side Encryption with Amazon S3-Managed Keys (SSE-S3), Server-Side Encryption with Customer Master Keys (CMKs) Stored in AWS Key Management Service (SSE-KMS), Server-Side Encryption with Customer-Provided Keys (SSE-C).

When you use server-side encryption with AWS KMS (SSE-KMS), you can use the default AWS managed CMK, or you can specify a customer managed CMK that you have already created. If you don't specify a customer managed CMK, Amazon S3 automatically creates an AWS managed CMK in your AWS account the first time that you add an object encrypted with SSE-KMS to a bucket. By default, Amazon S3 uses this CMK for SSE-KMS.

You can choose to have AWS KMS automatically rotate CMKs every year, provided that those keys were generated within AWS KMS HSMs.

Incorrect options:

Encrypt the data before sending it to Amazon S3 - The act of encrypting data before sending it to Amazon S3 is called Client-Side encryption. You will have to handle the key generation, maintenance and rotation process. Hence, this is not the right choice here.

Import a custom key into AWS KMS and automate the key rotation on an annual basis by using a Lambda function - When you import a custom key, you are responsible for maintaining a copy of your imported keys in your key management infrastructure so that you can re-import them at any time. Also, automatic key rotation is not supported for imported keys. Using Lambda functions to rotate keys is a possible solution, but not an optimal one for the current use case.

Use SSE-C with automatic key rotation on an annual basis - With Server-Side Encryption with Customer-Provided Keys (SSE-C), you manage the encryption keys and Amazon S3 manages the encryption, as it writes to disks, and decryption, when you access your objects. The keys are not stored anywhere in Amazon S3. There is no automatic key rotation facility for this option.

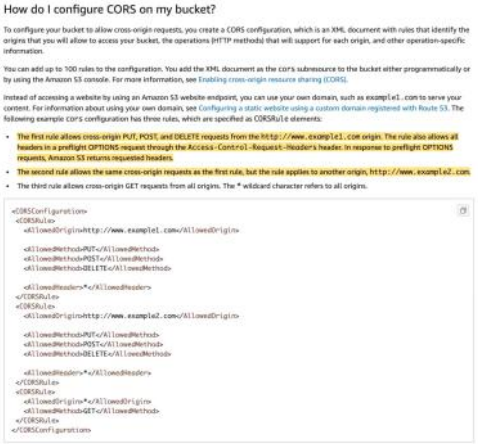
Question 30

You are storing your video files in a separate S3 bucket than your main static website in an S3 bucket. When accessing the video URLs directly the users can view the videos on the browser, but they can't play the videos while visiting the main website.

What is the root cause of this problem?

- Disable Server-Side Encryption
- change the bucket policy
- Amend the IAM policy
- Enable CORS

Overall explanation
Correct option:
Enable CORS
Cross-origin resource sharing (CORS) defines a way for client web applications that are loaded in one domain to interact with resources in a different domain. With CORS support, you can build rich client-side web applications with Amazon S3 and selectively allow cross-origin access to your Amazon S3 resources.
To configure your bucket to allow cross-origin requests, you create a CORS configuration, which is an XML document with rules that identify the origins that you will allow to access your bucket, the operations (HTTP methods) that will support for each origin, and other operation-specific information.
For the given use-case, you would create a <CORSRule> in <CORSConfiguration> for bucket B to allow access from the S3 website origin hosted on bucket A.
Please see this note for more details:

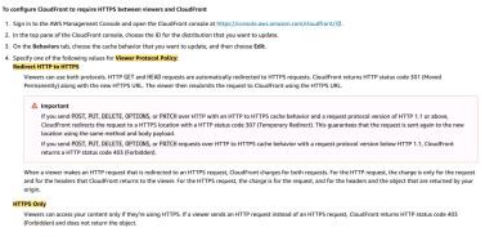


via - https://docs.aws.amazon.com/AmazonS3/latest/dev/cors.html
Incorrect options:
Change the bucket policy - A bucket policy is a resource-based AWS Identity and Access Management (IAM) policy that grants permissions. With this policy, you can do things such as allow one IP address to access the video file in the S3 bucket. In this scenario, we know that's not the case because it works using the direct URL but it doesn't work when you click on a link to access the video.
Amend the IAM policy - You attach IAM policies to IAM users, groups, or roles, which are then subject to the permissions you've defined. This scenario refers to public users of a website and they need not have an IAM user account.
Disable Server-Side Encryption - Amazon S3 encrypts your data at the object level as it writes it to disks in its data centers and decrypts it for you when you access it, if the video file is encrypted at rest then there is nothing you need to do because AWS handles encrypt and decrypt. Disabling encryption is not an issue because you can access the video directly using an URL but not from the main website.

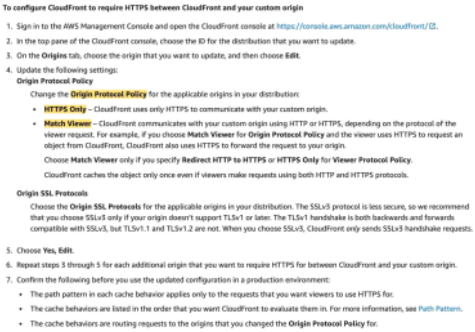
Question 31

Your company leverages Amazon CloudFront to provide content via the internet to customers with low latency. Aside from latency, security is another concern and you are looking for help in enforcing end-to-end connections using HTTPS so that content is protected.
Which of the following options is available for HTTPS in AWS CloudFront?
- Between clients and CloudFront only
- Neither between clients and CloudFront nor between CloudFront and backend
- Between clients and CloudFront as well as between CloudFront and backend
- Between CloudFront and backend only

Overall explanation
Correct option:
Between clients and CloudFront as well as between CloudFront and backend
For web distributions, you can configure CloudFront to require that viewers use HTTPS to request your objects, so connections are encrypted when CloudFront communicates with viewers.
Requiring HTTPS for Communication Between Viewers and CloudFront:



via - https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https-viewers-to-cloudfront.html
You also can configure CloudFront to use HTTPS to get objects from your origin, so connections are encrypted when CloudFront communicates with your origin.
Requiring HTTPS for Communication Between CloudFront and Your Custom Origin:



via - https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https-cloudfront-to-custom-origin.html
Incorrect options:
Between clients and CloudFront only - This is incorrect as you can choose to require HTTPS between CloudFront and your origin.
Between CloudFront and backend only - This is incorrect as you can choose to require HTTPS between viewers and CloudFront.
Neither between clients and CloudFront nor between CloudFront and backend - This is incorrect as you can choose HTTPS settings both for communication between viewers and CloudFront as well as between CloudFront and your origin.

Question 32

You are a manager for a tech company that has just hired a team of developers to work on the company's AWS infrastructure. All the developers are reporting to you that when using the AWS CLI to execute commands it fails with the following exception: You are not authorized to perform this operation. Encoded authorization failure message: 6h34GtpmGjJJUm946eDVBfzWQJk6z5GePbbGDs9Z2T8xZj9EZtEduSnTbmrR7pMqpJrVYJCew2m8YBZQf4HRWEtrpncANrZMsnzk.
Which of the following actions will help developers decode the message?
Your answer is correct

- AWS STS decode-authorization-message
- AWS Cognito Decoder
- Use KMS decode-authorization-message
- AWS IAM decode-authorization-message

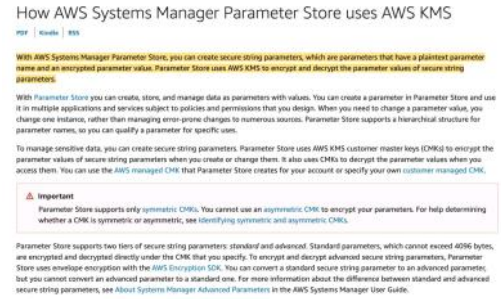
Overall explanation
Correct option:
AWS STS decode-authorization-message
Use decode-authorization-message to decode additional information about the authorization status of a request from an encoded message returned in response to an AWS request. If a user is not authorized to perform an action that was requested, the request returns a Client.UnauthorizedOperation response (an HTTP 403 response). The message is encoded because the details of the authorization status can constitute privileged information that the user who requested the operation should not see. To decode an authorization status message, a user must be granted permissions via an IAM policy to request the DecodeAuthorizationMessage (sts:DecodeAuthorizationMessage) action.
Incorrect options:
AWS IAM decode-authorization-message - The IAM service does not have this command, as it's a made-up option.
Use KMS decode-authorization-message - The KMS service does not have this command, as it's a made-up option.
AWS Cognito Decoder - The Cognito service does not have this command, as it's a made-up option.
Reference:
https://docs.aws.amazon.com/cli/latest/reference/sts/decode-authorization-message.html

Question 33

You are planning to build a fleet of EBS-optimized EC2 instances to handle the load of your new application. Due to security compliance, your organization wants any secret strings used in the application to be encrypted to prevent exposing values as clear text.
The solution requires that decryption events be audited and API calls to be simple. How can this be achieved? (select two)

- Store the secret as SecureString in SSM Parameter Store
- Encrypt first with KMS then store in SSM Parameter store
- Audit using CloudTrail
- Audit using SSM Audit Trail

Store the secret as PlainText in SSM Parameter Store
Overall explanation
Correct options:
Store the secret as SecureString in SSM Parameter Store
With AWS Systems Manager Parameter Store, you can create SecureString parameters, which are parameters that have a plaintext parameter name and an encrypted parameter value. Parameter Store uses AWS KMS to encrypt and decrypt the parameter values of Secure String parameters. Also, if you are using customer-managed CMKs, you can use IAM policies and key policies to manage to encrypt and decrypt permissions. To retrieve the decrypted value you only need to do one API call.



via - https://docs.aws.amazon.com/kms/latest/developerguide/services-parameter-store.html
Audit using CloudTrail
AWS CloudTrail is a service that enables governance, compliance, operational auditing, and risk auditing of your AWS account. With CloudTrail, you can log, continuously monitor, and retain account activity related to actions across your AWS infrastructure. CloudTrail provides an event history of your AWS account activity, including actions taken through the AWS Management Console, AWS SDKs, command-line tools, and other AWS services.
CloudTrail will allow you to see all API calls made to SSM and KMS.

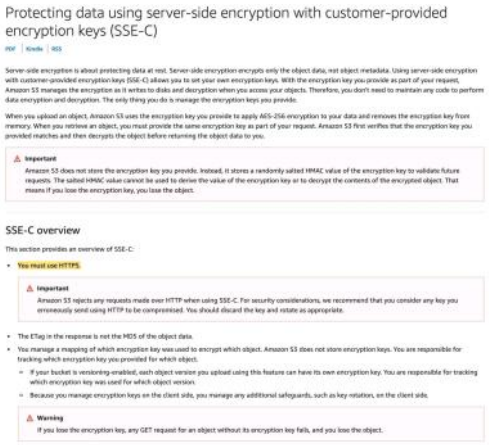
Incorrect options:
Encrypt first with KMS then store in SSM Parameter store - This could work but will require two API calls to get the decrypted value instead of one. So this is not the right option.
Store the secret as PlainText in SSM Parameter Store - Plaintext parameters are not secure and shouldn't be used to store secrets.
Audit using SSM Audit Trail - This is a made-up option and has been added as a distractor.

Question 34

As part of internal regulations, you must ensure that all communications to Amazon S3 are encrypted.
For which of the following encryption mechanisms will a request get rejected if the connection is not using HTTPS?

SSE-KMS
SSE-S3
Client Side Encryption
SSE-C
Overall explanation

Correct option:
SSE-C
Server-side encryption is about protecting data at rest. Server-side encryption encrypts only the object data, not object metadata. Using server-side encryption with customer-provided encryption keys (SSE-C) allows you to set your encryption keys.
When you upload an object, Amazon S3 uses the encryption key you provide to apply AES-256 encryption to your data and removes the encryption key from memory. When you retrieve an object, you must provide the same encryption key as part of your request. Amazon S3 first verifies that the encryption key you provided matches and then decrypts the object before returning the object data to you.
Amazon S3 will reject any requests made over HTTP when using SSE-C. For security considerations, AWS recommends that you consider any key you send erroneously using HTTP to be compromised.



Protecting data using server-side encryption with customer-provided encryption keys (SSE-C)

PDF | Kindle | RSS

Server-side encryption is about protecting data at rest. Server-side encryption encrypts only the object data, not object metadata. Using server-side encryption with customer-provided encryption keys (SSE-C) allows you to set your own encryption keys. With the encryption key you provide as part of your request, Amazon S3 manages the encryption as it writes to disks and decryption when you access your objects. Therefore, you don't need to maintain any code to perform data encryption and decryption. The only thing you do is manage the encryption keys you provide.

When you upload an object, Amazon S3 uses the encryption key you provide to apply AES-256 encryption to your data and removes the encryption key from memory. When you retrieve an object, you must provide the same encryption key as part of your request. Amazon S3 first verifies that the encryption key you provided matches and then decrypts the object before returning the object data to you.

Important

Amazon S3 does not store the encryption key you provide. Instead, it stores a randomly salted HMAC value of the encryption key to validate future requests. The salted HMAC value cannot be used to derive the value of the encryption key or to decrypt the contents of the encrypted object. That means if you lose the encryption key, you lose the object.

SSE-C overview

This section provides an overview of SSE-C:

- You must use HTTPS.

Important

Amazon S3 rejects any requests made over HTTP when using SSE-C. For security considerations, we recommend that you consider any key you erroneously send using HTTP to be compromised. You should discard the key and rotate as appropriate.

- The ETag in the response is not the MD5 of the object data.
- You manage a mapping of which encryption key was used to encrypt which object. Amazon S3 does not store encryption keys. You are responsible for tracking which encryption key you provided for which object.
  - If your bucket is versioning-enabled, each object version you upload using this feature can have its own encryption key. You are responsible for tracking which encryption key was used for which object version.
  - Because you manage encryption keys on the client side, you manage any additional safeguards, such as key rotation, on the client side.

Warning

If you lose the encryption key, any GET request for an object without its encryption key fails, and you lose the object.

via - https://docs.aws.amazon.com/AmazonS3/latest/dev/ServerSideEncryptionCustomerKeys.html

Incorrect options:

SSE-KMS - It is not mandatory to use HTTPS.

Client-Side Encryption - Client-side encryption is the act of encrypting data before sending it to Amazon S3. It is not mandatory to use HTTPS for this.

SSE-S3 - It is not mandatory to use HTTPS. Amazon S3 encrypts your data at the object level as it writes it to disks in its data centers.

Question 35

A company has a new media application that utilizes an Amazon CloudFront distribution that accesses the S3 bucket by using an origin access identity (OAI). The S3 bucket has an explicit access denial for all other users. A developer wants to allow access to the login page for unauthenticated users while ensuring the security of all private content that has restricted viewer access.
Which of the following will you recommend?

- Configure a new distribution having the same origin as the original distribution and set the path pattern for the default cache behavior of the new distribution as the path of the login page with viewer access as unrestricted. Keep the default cache behavior of the original distribution unchanged
- Configure a second cache behavior to the distribution having the same origin as the default cache behavior and have the path pattern for the second cache behavior as the path of the login page with viewer access as unrestricted. Keep the default cache behavior's settings unchanged
- Configure a second cache behavior to the distribution having the same origin as the default cache behavior and have the path pattern for the second cache behavior as * with viewer access as restricted. Modify the default cache behavior's path pattern to the path of the login page and have the viewer access as unrestricted
- Configure a second origin as the failover origin for the default behavior of the original distribution and have the path pattern for the second origin as the path of the login page with viewer access as unrestricted. Keep the behavior for the primary origin unchanged

Overall explanation

Correct option:

Configure a second cache behavior to the distribution having the same origin as the default cache behavior and have the path pattern for the second cache behavior as the path of the login page with viewer access as unrestricted. Keep the default cache behavior's settings unchanged

Cache behavior describes how CloudFront processes requests. You must create at least as many cache behaviors (including the default cache behavior) as you have origins if you want CloudFront to serve objects from all of the origins. Each cache behavior specifies the one origin from which you want CloudFront to get objects. If you have two origins and only the default cache behavior, the default cache behavior will cause CloudFront to get objects from one of the origins, but the other origin is never used.

The pattern (for example, images/*.jpg) specifies which requests to apply the behavior to. When CloudFront receives a viewer request, the requested path is compared with path patterns in the order in which cache behaviors are listed in the distribution. The path pattern for the default cache behavior is * and cannot be changed. If the request for an object does not match the path pattern for any cache behaviors, CloudFront applies the behavior in the default cache behavior.

For the given use case, you need to add a second cache behavior to the distribution having the same origin as the default cache behavior and list it above the default cache behavior in the distribution. The second cache behavior should have the path pattern as the path of the login page with viewer access set as unrestricted. This would allow access to the login page for unauthenticated users. Since the default cache behavior's settings remain unchanged, it ensures the security of all private content that continues to have restricted viewer access.

Incorrect options:

Configure a second cache behavior to the distribution having the same origin as the default cache behavior and have the path pattern for the second cache behavior as * with viewer access as restricted. Modify the default cache behavior's path pattern to the path of the login page and have the viewer access as unrestricted - This option is incorrect since the path pattern for the default cache behavior is always * and cannot be changed.

Configure a new distribution having the same origin as the original distribution and set the path pattern for the default cache behavior of the new distribution as the path of the login page with viewer access as unrestricted. Keep the default cache behavior of the original distribution unchanged - If you have two origins and only the default cache behavior, the default cache behavior will cause CloudFront to get objects from one of the origins, but the other origin is never used. So this option is incorrect.

Configure a second origin as the failover origin for the default behavior of the original distribution and have the path pattern for the second origin as the path of the login page with viewer access as unrestricted. Keep the behavior for the primary origin unchanged - You can set up CloudFront with origin failover for scenarios that require high availability. To get started, you create an origin group with two origins: a primary and a secondary. If the primary origin is unavailable or returns specific HTTP response status codes that indicate a failure, CloudFront automatically switches to the secondary origin. To set up origin failover, you must have a distribution with at least two origins. Next, you create an origin group for your distribution that includes two origins, setting one as the primary. Finally, you create or update a cache behavior to use the origin group.

This option is incorrect since the failover kicks in only when the primary is unavailable. Therefore, access to the login page and the rest of the content will never work together.

Question 36

A financial services company uses Amazon S3 to store transformed and anonymized customer data that is generated by a daily batch job. The development team has been tasked to build a solution that analyzes the output of the daily job for any sensitive financial information about the company's customers.
As an AWS Certified Developer Associate, which of the following options would you recommend to address this use case MOST efficiently?

- Leverage Macie to analyze the output of the daily batch job and look for any sensitive data findings of type SensitiveData:S3Object/CustomIdentifier
- Leverage Macie to analyze the output of the daily batch job and look for any sensitive data findings of type SensitiveData:S3Object/Personal
- Configure a S3 event notification for every object upload that triggers a Lambda function based Python script to detect sensitive customer information
- Leverage Macie to analyze the output of the daily batch job and look for any sensitive data findings of type SensitiveData:S3Object/Financial

Overall explanation

Correct option:

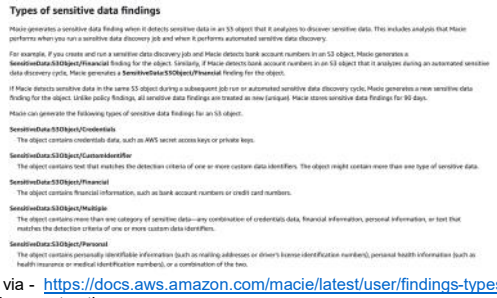
Leverage Macie to analyze the output of the daily batch job and look for any sensitive data findings of type SensitiveData:S3Object/Financial

Amazon Macie is a data security service that discovers sensitive data by using machine learning and pattern matching, provides visibility into data security risks, and enables automated protection against those risks. To help you manage the security posture of your organization's Amazon Simple Storage Service (Amazon S3) data estate, Macie provides you with an inventory of your S3 buckets, and automatically evaluates and monitors the buckets for security and access control. If Macie detects a potential issue with the security or privacy of your data, such as a bucket that becomes publicly accessible, Macie generates a finding for you to review and remediate as necessary.

Macie also automates the discovery and reporting of sensitive data to provide you with a better understanding of the data that your organization stores in Amazon S3. To detect sensitive data, you can use built-in criteria and techniques that Macie provides, custom criteria that you define, or a combination of the two. If Macie detects sensitive data in an S3 object, Macie generates a finding to notify you of the sensitive data that Macie found.

Macie generates a sensitive data finding when it detects sensitive data in an S3 object that it analyzes to discover sensitive data. This includes analysis that Macie performs when you run a sensitive data discovery job and when it performs automated sensitive data discovery.

For the given use case, you can use Macie to analyze the output of the daily batch job and look for any sensitive data findings of type SensitiveData:S3Object/Financial which implies that the S3 object contains financial information, such as bank account numbers or credit card numbers.



Types of sensitive data findings

Macie generates a sensitive data finding when it detects sensitive data in an S3 object that it analyzes to discover sensitive data. This includes analysis that Macie performs when you run a sensitive data discovery job and when it performs automated sensitive data discovery.

For example, if you create and run a sensitive data discovery job and Macie detects bank account numbers in an S3 object, Macie generates a **SensitiveData:S3Object/Financial** finding for the object. Similarly, if Macie detects bank account numbers in an S3 object that it analyzes during an automated sensitive data discovery cycle, Macie generates a **SensitiveData:S3Object/Financial** finding for the object.

If Macie detects sensitive data in the same S3 object during a subsequent job run or automated sensitive data discovery cycle, Macie generates a new sensitive data finding for the object. Unlike policy findings, all sensitive data findings are treated as new (unique). Macie stores sensitive data findings for 90 days.

Macie can generate the following types of sensitive data findings for an S3 object.

SensitiveData:S3Object/Credentials
The object contains credentials data, such as AWS secret access keys or private keys.

SensitiveData:S3Object/CustomIdentifier
The object contains text that matches the detection criteria of one or more custom data identifiers. The object might contain more than one type of sensitive data.

SensitiveData:S3Object/Financial
The object contains financial information, such as bank account numbers or credit card numbers.

SensitiveData:S3Object/Multiple
The object contains more than one category of sensitive data—any combination of credentials data, financial information, personal information, or text that matches the detection criteria of one or more custom data identifiers.

SensitiveData:S3Object/Personal
The object contains personally identifiable information (such as mailing addresses or driver's license identification numbers), personal health information (such as health insurance or medical identification numbers), or a combination of the two.

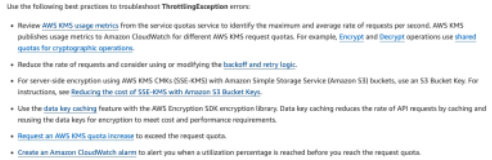
via - https://docs.aws.amazon.com/macie/latest/user/findings-types.html
Incorrect options:
Leverage Macie to analyze the output of the daily batch job and look for any sensitive data findings of type SensitiveData:S3Object/Personal - The object contains personally identifiable information (such as mailing addresses or driver's license identification numbers), personal health information (such as health insurance or medical identification numbers), or a combination of the two.
Leverage Macie to analyze the output of the daily batch job and look for any sensitive data findings of type SensitiveData:S3Object/CustomIdentifier - The object contains text that matches the detection criteria of one or more custom data identifiers. The object might contain more than one type of sensitive data.
As explained above, for the given use case, you need to look for any sensitive data findings of type SensitiveData:S3Object/Financial. So both these options are incorrect.
Configure a S3 event notification for every object upload that triggers a Lambda function based custom application to detect sensitive customer information - Maintaining a custom application to detect sensitive information would be highly inefficient as you need to consistently upgrade the application to handle new formats and types of financial information. This is better handled by leveraging Macie's Findings.

Question 37

An e-commerce application posts its order transactions in bulk to an accounting application for further processing. Due to changes in the compliance rules, all the transactions are being encrypted with AWS Key Management Service (AWS KMS) key before posting to the accounting application. Post this change, the testers have raised tickets regarding the application receiving a ThrottlingException error.
What measures should a developer take to fix this issue MOST optimally? (Select two)

- Reduce the rate of requests and consider using the backoff and retry logic
- Write queries in Amazon CloudWatch Logs Insights to track your API request usage and submit an AWS Support case to request a quota increase
- Use the data key caching feature with the AWS Encryption SDK encryption library
- Use a bucket-level key for SSE-KMS which will decrease the requested traffic to AWS KMS thereby avoiding the ThrottlingException error

Send AWS CloudTrail events generated by AWS KMS to Amazon CloudWatch Logs
Overall explanation
Correct options:
Reduce the rate of requests and consider using the backoff and retry logic
Each AWS SDK implements automatic retry logic. The AWS SDK for Java automatically retries requests, and you can configure the retry settings. In addition to simple retries, each AWS SDK implements an exponential backoff algorithm for better flow control. The idea behind exponential backoff is to use progressively longer waits between retries for consecutive error responses. You should implement a maximum delay interval, as well as a maximum number of retries. The maximum delay interval and the maximum number of retries are not necessarily fixed values and should be set based on the operation being performed, as well as other local factors, such as network latency.
Use the data key caching feature with the AWS Encryption SDK encryption library
Data key caching stores data keys and related cryptographic material in a cache. When you encrypt or decrypt data, the AWS Encryption SDK looks for a matching data key in the cache. If it finds a match, it uses the cached data key rather than generating a new one. Data key caching can improve performance, reduce cost, and help you stay within service limits as your application scales. Your application can benefit from data key caching if: 1. It can reuse data keys. 2. It generates numerous data keys. 3. Your cryptographic operations are unacceptably slow, expensive, limited, or resource-intensive.
Data key caching is an optional feature of the AWS Encryption SDK that you should use cautiously. By default, the AWS Encryption SDK generates a new data key for every encryption operation. This technique supports cryptographic best practices, which discourage excessive reuse of data keys. In general, use data key caching only when it is required to meet your performance goals. Then, use the data key caching security thresholds to ensure that you use the minimum amount of caching required to meet your cost and performance goals.
Best practices to troubleshoot ThrottlingException errors:



Use the following best practices to troubleshoot **ThrottlingException** errors:

- Review AWS KMS usage metrics from the service quotas service to identify the maximum and average rate of requests per second. AWS KMS publishes usage metrics to Amazon CloudWatch for different AWS KMS request quotas. For example, Encrypt and Decrypt operations use shared quotas for cryptographic operations.
- Reduce the rate of requests and consider using or modifying the backoff and retry logic.
- For server-side encryption using AWS KMS CMKs (SSE-KMS) with Amazon Simple Storage Service (Amazon S3) buckets, use an S3 Bucket Key. For instructions, see Reducing the cost of SSE-KMS with Amazon S3 Bucket Keys.
- Use the data key caching feature with the AWS Encryption SDK encryption library. Data key caching reduces the rate of API requests by caching and reusing the data keys for encryption to meet cost and performance requirements.
- Request an AWS KMS quota increase to exceed the request quota.
- Create an Amazon CloudWatch alarm to alert you when a utilization percentage is reached before you reach the request quota.

via - https://aws.amazon.com/premiumsupport/knowledge-center/kms-throttlingexception-error/
Incorrect options:
Send AWS CloudTrail events generated by AWS KMS to Amazon CloudWatch Logs - Deeper analysis of CloudTrail data sent to CloudWatch logs can help spot throttled API calls. While it is certainly possible to track API usage using CloudTrail data, it is not an optimal way to fix the ThrottlingException error.
Write queries in Amazon CloudWatch Logs Insights to track your API request usage and submit an AWS Support case to request a quota increase - Historically, to understand how close to a request rate quota you were, you had to perform three tasks: (i) send AWS CloudTrail events generated by AWS KMS to Amazon CloudWatch Logs; (ii) write queries in Amazon CloudWatch Logs Insights to track your API request usage; and (iii) submit an AWS Support case to request a quota increase. Now, you can view your AWS KMS API usage and request quota increases within the AWS Service Quotas console itself without doing any special configuration.
Use a bucket-level key for SSE-KMS which will decrease the requested traffic to AWS KMS thereby avoiding the ThrottlingException error - Amazon S3 bucket has not been mentioned in the given use case and hence this option is irrelevant.

Question 38

You would like your Elastic Beanstalk environment to expose an HTTPS endpoint and an HTTP endpoint. The HTTPS endpoint should be used to get in-flight encryption between your clients and your web servers, while the HTTP endpoint should only be used to redirect traffic to HTTPS and support URLs starting with http://.
What must be done to configure this setup? (Select three)

Only open up port 80
Configure your EC2 instances to redirect HTTPS traffic to HTTP
Assign an SSL certificate to the Load Balancer
Only open up port 443
Configure your EC2 instances to redirect HTTP traffic to HTTPS
Open up port 80 & port 443

Overall explanation
Correct options:
Assign an SSL certificate to the Load Balancer
This ensures that the Load Balancer can expose an HTTPS endpoint.
Open up port 80 & port 443
This ensures that the Load Balancer will allow both the HTTP (80) and HTTPS (443) protocol for incoming connections
Configure your EC2 instances to redirect HTTP traffic to HTTPS
This ensures traffic originating from HTTP onto the Load Balancer forces a redirect to HTTPS by the EC2 instances before being correctly served, thus ensuring the traffic served is fully encrypted.
Incorrect options:
Only open up port 80 - This is not correct as it would not allow HTTPS traffic (port 443).
Only open up port 443 - This is not correct as it would not allow HTTP traffic (port 80).
Configure your EC2 instances to redirect HTTPS traffic to HTTP - This is not correct as it would force HTTP traffic, instead of HTTPS.

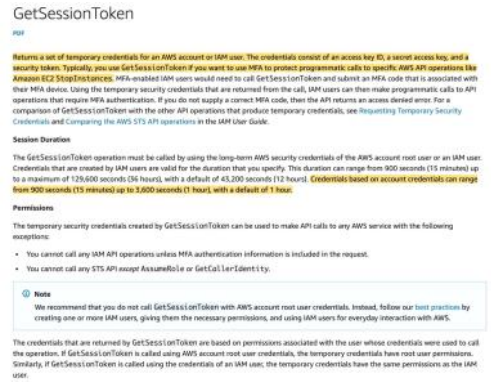
Question 39

A developer created an online shopping application that runs on EC2 instances behind load balancers. The same web application version is hosted on several EC2 instances and the instances run in an Auto Scaling group. The application uses STS to request credentials but after an hour your application stops working.
What is the most likely cause of this issue?

- Your IAM policy is wrong

- A lambda function revokes your access every hour
- The IAM service is experiencing downtime once an hour
- Your application needs to renew the credentials after 1 hour when they expire

Overall explanation
Correct option:
Your application needs to renew the credentials after 1 hour when they expire
AWS Security Token Service (AWS STS) is a web service that enables you to request temporary, limited-privilege credentials for AWS Identity and Access Management (IAM) users or for users that you authenticate (federated users). By default, AWS Security Token Service (STS) is available as a global service, and all AWS STS requests go to a single endpoint at https://sts.amazonaws.com. Credentials that are created by using account credentials can range from 900 seconds (15 minutes) up to a maximum of 3,600 seconds (1 hour), with a default of 1 hour. Hence you need to renew the credentials post expiry.


GetSessionToken

Returns a set of temporary credentials for an AWS account or IAM user. The credentials consist of an access key ID, a secret access key, and a security token. Typically, you use GetSessionToken if you want to use MFA to protect programmatic calls to specific AWS API operations like Amazon EC2 StopInstances. MFA-enabled IAM users would need to call GetSessionToken and submit an MFA code that is associated with their MFA device. Using the temporary security credentials that are returned from the call, IAM users can then make programmatic calls to API operations that require MFA authentication. If you do not supply a correct MFA code, then the API returns an access denied error. For a comparison of GetSessionToken with the other API operations that produce temporary credentials, see Requesting Temporary Security Credentials and Comparing the AWS STS API operations in the IAM User Guide.

Session Duration

The GetSessionToken operation must be called by using the long-term AWS security credentials of the AWS account root user or an IAM user. Credentials that are created by IAM users are valid for the duration that you specify. This duration can range from 900 seconds (15 minutes) up to a maximum of 129,600 seconds (36 hours), with a default of 43,200 seconds (12 hours). Credentials based on account credentials can range from 900 seconds (15 minutes) up to 3,600 seconds (1 hour), with a default of 1 hour.

Permissions

The temporary security credentials created by GetSessionToken can be used to make API calls to any AWS service with the following exceptions:

- You cannot call any IAM API operations unless MFA authentication information is included in the request.
- You cannot call any STS API except AssumeRole or GetCallerIdentity.

Note
We recommend that you do not call GetSessionToken with AWS account root user credentials. Instead, follow our best practices by creating one or more IAM users, giving them the necessary permissions, and using IAM users for everyday interaction with AWS.

The credentials that are returned by GetSessionToken are based on permissions associated with the user whose credentials were used to call the operation. If GetSessionToken is called using AWS account root user credentials, the temporary credentials have root user permissions. Similarly, if GetSessionToken is called using the credentials of an IAM user, the temporary credentials have the same permissions as the IAM user.

via - https://docs.aws.amazon.com/STS/latest/APIReference/API_GetSessionToken.html
Incorrect options:
Your IAM policy is wrong - If your policy was wrong, a reboot would not solve the issue.
A lambda function revokes your access every hour - Revoking can be done by an IAM policy. Lambda function cannot revoke access.
The IAM service is experiencing downtime once an hour - The IAM service is reliable as it's managed by AWS.

Question 40

You've just deployed an AWS Lambda function. The lambda function will be invoked via the API Gateway. The API Gateway will need to control access to it.
Which of the following mechanisms is not supported for API Gateway?

- Cognito User Pools
- IAM permissions with sigv4
- STS
- Lambda Authorizer

Overall explanation
Correct option:
Amazon API Gateway is an AWS service for creating, publishing, maintaining, monitoring, and securing REST, HTTP, and WebSocket APIs at any scale. API developers can create APIs that access AWS or other web services, as well as data stored in the AWS Cloud.
How API Gateway Works:

via - https://aws.amazon.com/api-gateway/
STS
The AWS Security Token Service (STS) is a web service that enables you to request temporary, limited-privilege credentials for AWS Identity and Access Management (IAM) users or for users that you authenticate (federated users). However, is not supported at the time with API Gateway.
Incorrect options:
IAM permissions with sigv4 - They can be applied to an entire API or individual methods.
Lambda Authorizer - Control access to REST API methods using bearer token authentication as well as information described by headers, paths, query strings, stage variables, or context variables request parameter.
Cognito User Pools - Use Cognito User Pools to create customizable authentication and authorization solutions for your REST APIs.

Question 41

A security company is requiring all developers to perform server-side encryption with customer-provided encryption keys when performing operations in AWS S3. Developers should write software with C# using the AWS SDK and implement the requirement in the PUT, GET, Head, and Copy operations.
Which of the following encryption methods meets this requirement?

- SSE-S3
- SSE-KMS
- Client-Side Encryption
- SSE-C

Overall explanation
Correct option:
SSE-C
You have the following options for protecting data at rest in Amazon S3:
Server-Side Encryption – Request Amazon S3 to encrypt your object before saving it on disks in its data centers and then decrypt it when you download the objects.
Client-Side Encryption – Encrypt data client-side and upload the encrypted data to Amazon S3. In this case, you manage the encryption process, the encryption keys, and related tools.
For the given use-case, the company wants to manage the encryption keys via its custom application and let S3 manage the encryption, therefore you must use Server-Side Encryption with Customer-Provided Keys (SSE-C).
Using server-side encryption with customer-provided encryption keys (SSE-C) allows you to set your encryption keys. With the encryption key you provide as part of your request, Amazon S3 manages both the encryption, as it writes to disks, and decryption, when you access your objects.
Please review these three options for Server Side Encryption on S3:

Protecting data using server-side encryption

Server-side encryption is the encryption of data at its destination by the application or service that receives it. Amazon S3 encrypts your data at the object level as it writes it to disks in its data centers and decrypts it for you when you access it. As long as you authenticate your request and you have access permissions, there is no difference in the way you access encrypted or unencrypted objects. For example, if you share your objects using a presigned URL, that URL works the same way for both encrypted and unencrypted objects. Additionally, when you list objects in your bucket, the list API returns a list of all objects, regardless of whether they are encrypted.

Note
You can't apply different types of server-side encryption to the same object simultaneously.

You have three mutually exclusive options, depending on how you choose to manage the encryption keys.

Server-Side Encryption with Amazon S3-Managed Keys (SSE-S3)

When you use Server-Side Encryption with Amazon S3-Managed Keys (SSE-S3), each object is encrypted with a unique key. As an additional safeguard, it encrypts the key itself with a master key that it regularly rotates. Amazon S3 server-side encryption uses one of the strongest block ciphers available, 256-bit Advanced Encryption Standard (AES-256), to encrypt your data. For more information, see Protecting Data Using Server-Side Encryption with Amazon S3-Managed Encryption Keys (SSE-S3).

Server-Side Encryption with Customer Master Keys (CMKs) Stored in AWS Key Management Service (SSE-KMS)

Server-Side Encryption with Customer Master Keys (CMKs) Stored in AWS Key Management Service (SSE-KMS) is similar to SSE-S3, but with some additional benefits and charges for using this service. There are separate permissions for the use of a CMK that provides added protection against unauthorized access of your objects in Amazon S3. SSE-KMS also provides you with an audit trail that shows when your CMK was used and by whom. Additionally, you can create and manage customer managed CMKs or use AWS managed CMKs that are unique to you, your service, and your Region. For more information, see Protecting Data Using Server-Side Encryption with CMKs Stored in AWS Key Management Service (SSE-KMS).

Server-Side Encryption with Customer-Provided Keys (SSE-C)

With Server-Side Encryption with Customer-Provided Keys (SSE-C), you manage the encryption keys and Amazon S3 manages the encryption, as it writes to disks, and decryption, when you access your objects. For more information, see Protecting data using server-side encryption with customer-provided encryption keys (SSE-C).

via - https://docs.aws.amazon.com/AmazonS3/latest/dev/serv-side-encryption.html
Incorrect options:
SSE-KMS - Server-Side Encryption with Customer Master Keys (CMKs) stored in AWS Key Management Service (SSE-KMS) is similar to SSE-S3. SSE-KMS provides you with an audit trail that shows when your CMK was used and by whom. Additionally, you can create and manage customer-managed CMKs or use AWS managed CMKs that are unique to you, your service, and your Region.
Client-Side Encryption - You can encrypt the data client-side and upload the encrypted data to Amazon S3. In this case, you manage the encryption process, the encryption keys, and related tools.
SSE-S3 - When you use Server-Side Encryption with Amazon S3-Managed Keys (SSE-S3), each object is encrypted with a unique key. As an additional safeguard, it encrypts the key itself with a master key that it regularly rotates. So this option is incorrect.

Question 42

You were assigned to a project that requires the use of the AWS CLI to build a project with AWS CodeBuild. Your project's root directory includes the buildspec.yml file to run build commands and would like your build artifacts to be automatically encrypted at the end.
How should you configure CodeBuild to accomplish this?

- Specify a KMS key to use
- Use In Flight encryption (SSL)
- Use the AWS Encryption SDK
- Use an AWS Lambda Hook

Overall explanation
Correct option:
Specify a KMS key to use
AWS Key Management Service (KMS) makes it easy for you to create and manage cryptographic keys and control their use across a wide range of AWS services and in your applications.
For AWS CodeBuild to encrypt its build output artifacts, it needs access to an AWS KMS customer master key (CMK). By default, AWS CodeBuild uses the AWS-managed CMK for Amazon S3 in your AWS account. The following environment variable provides these details:
CODEBUILD_KMS_KEY_ID: The identifier of the AWS KMS key that CodeBuild is using to encrypt the build output artifact (for example, arn:aws:kms:region-ID:account-ID:key/key-ID or alias/key-alias).
Incorrect options:
Use an AWS Lambda Hook - Code hook is used for integration with Lambda and is not relevant for the given use-case.
Use the AWS Encryption SDK - The SDK just makes it easier for you to implement encryption best practices in your application and is not relevant for the given use-case.
Use In-Flight encryption (SSL) - SSL is usually for internet traffic which in this case will be using internal traffic through AWS and is not relevant for the given use-case.

Question 43

An e-commerce company has multiple EC2 instances operating in a private subnet which is part of a custom VPC. These instances are running an image processing application that needs to access images stored on S3. Once each image is processed, the status of the corresponding record needs to be marked as completed in a DynamoDB table.
How would you go about providing private access to these AWS resources which are not part of this custom VPC?

- Create a gateway endpoint for DynamoDB and add it as a target in the route table of the custom VPC. Create an API endpoint for S3 and then connect to the S3 service using the private IP address
- Create a separate gateway endpoint for S3 and DynamoDB each. Add two new target entries for these two gateway endpoints in the route table of the custom VPC
- Create a separate interface endpoint for S3 and DynamoDB each. Then connect to these services using the private IP address
- Create a gateway endpoint for S3 and add it as a target in the route table of the custom VPC. Create an interface endpoint for DynamoDB and then connect to the DynamoDB service using the private IP address

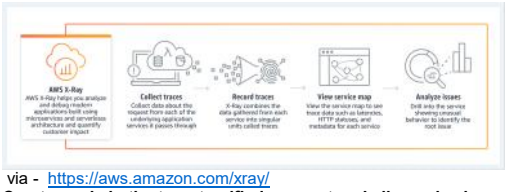
Overall explanation
Correct option:
Create a separate gateway endpoint for S3 and DynamoDB each. Add two new target entries for these two gateway endpoints in the route table of the custom VPC
Endpoints are virtual devices. They are horizontally scaled, redundant, and highly available VPC components. They allow communication between instances in your VPC and services without imposing availability risks or bandwidth constraints on your network traffic.
A VPC endpoint enables you to privately connect your VPC to supported AWS services and VPC endpoint services powered by AWS PrivateLink without requiring an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. Instances in your VPC do not require public IP addresses to communicate with resources in the service. Traffic between your VPC and the other service does not leave the Amazon network.
There are two types of VPC endpoints: interface endpoints and gateway endpoints. An interface endpoint is an elastic network interface with a private IP address from the IP address range of your subnet that serves as an entry point for traffic destined to a supported service.
A gateway endpoint is a gateway that you specify as a target for a route in your route table for traffic destined to a supported AWS service. The following AWS services are supported:
Amazon S3
DynamoDB
You should note that S3 now supports both gateway endpoints as well as the interface endpoints.
Incorrect options:
Create a gateway endpoint for S3 and add it as a target in the route table of the custom VPC. Create an interface endpoint for DynamoDB and then connect to the DynamoDB service using the private IP address
Create a separate interface endpoint for S3 and DynamoDB each. Then connect to these services using the private IP address
DynamoDB does not support interface endpoints, so these two options are incorrect.
Create a gateway endpoint for DynamoDB and add it as a target in the route table of the custom VPC. Create an API endpoint for S3 and then connect to the S3 service using the private IP address - There is no such thing as an API endpoint for S3. API endpoints are used with AWS API Gateway. This option has been added as a distractor.

Question 44

Your company likes to operate multiple AWS accounts so that teams have their environments. Services deployed across these accounts interact with one another, and now there's a requirement to implement X-Ray traces across all your applications deployed on EC2 instances and AWS accounts.
As such, you would like to have a unified account to view all the traces. What should you in your X-Ray daemon set up to make this work? (Select two)

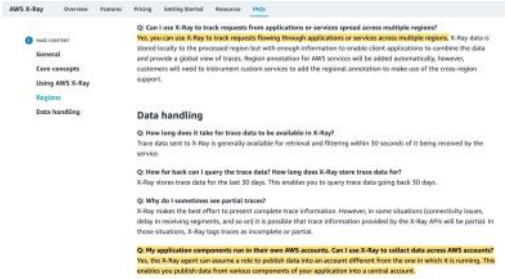
- Configure the X-Ray daemon to use access and secret keys
- Create a user in the target unified account and generate access and secret keys
- Configure the X-Ray daemon to use an IAM instance role
- Enable Cross Account collection in the X-Ray console
- Create a role in the target unified account and allow roles in each sub-account to assume the role.

Overall explanation
Correct option:
AWS X-Ray helps developers analyze and debug production, distributed applications, such as those built using a microservices architecture. With X-Ray, you can understand how your application and its underlying services are performing to identify and troubleshoot the root cause of performance issues and errors. X-Ray provides an end-to-end view of requests as they travel through your application, and shows a map of your application's underlying components.
How X-Ray Works:



via - https://aws.amazon.com/xray/

Create a role in the target unified account and allow roles in each sub-account to assume the role
Configure the X-Ray daemon to use an IAM instance role
The X-Ray agent can assume a role to publish data into an account different from the one in which it is running. This enables you to publish data from various components of your application into a central account.
X-Ray can also track requests flowing through applications or services across multiple AWS Regions.



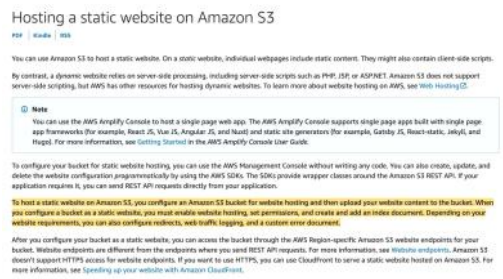
via - https://aws.amazon.com/xray/faqs/
You can create the necessary configurations for cross-account access via this reference documentation - https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon-configuration.html
Incorrect options:
Create a user in the target unified account and generate access and secret keys
Configure the X-Ray daemon to use access and secret keys
These two options combined together would work but wouldn't be a best-practice security-wise. Therefore these are not correct.
Enable Cross Account collection in the X-Ray console - This is a made-up option and has been added as a distractor.

Question 45

Your company is new to cloud computing and would like to host a static HTML5 website on the cloud and be able to access it via domain www.mycompany.com. You have created a bucket in Amazon Simple Storage Service (S3), enabled website hosting, and set the index.html as the default page. Finally, you create an Alias record in Amazon Route 53 that points to the S3 website endpoint of your S3 bucket.
When you test the domain www.mycompany.com you get the following error: 'HTTP response code 403 (Access Denied)'. What can you do to resolve this error?

- Enable Encryption
- Create a bucket policy
- Create an IAM role
- Enable CORS

Overall explanation
Correct answer
Create a bucket policy
Bucket policy is an access policy option available for you to grant permission to your Amazon S3 resources. It uses JSON-based access policy language.
If you want to configure an existing bucket as a static website that has public access, you must edit block public access settings for that bucket. You may also have to edit your account-level block public access settings.



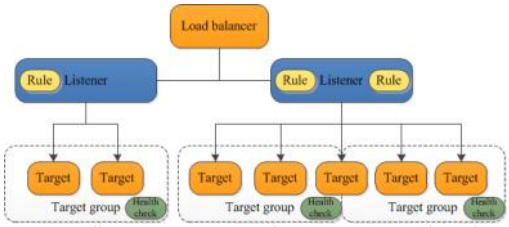
via - https://docs.aws.amazon.com/AmazonS3/latest/dev/WebsiteHosting.html
Incorrect:
Create an IAM role - This will not help because IAM roles are attached to services and in this case, we have public users.
Enable CORS - CORS defines a way for client web applications that are loaded in one domain to interact with resources in a different domain. Here we are not dealing with cross domains.
Enable Encryption - For the most part, encryption does not have an effect on access denied/forbidden errors. On the website endpoint, if a user requests an object that doesn't exist, Amazon S3 returns HTTP response code 404 (Not Found). If the object exists but you haven't granted read permission on it, the website endpoint returns HTTP response code 403 (Access Denied).

Question 46

An e-commerce company has a fleet of EC2 based web servers running into very high CPU utilization issues. The development team has determined that serving secure traffic via HTTPS is a major contributor to the high CPU load.
Which of the following steps can take the high CPU load off the web servers? (Select two)

- Configure an SSL/TLS certificate on an Application Load Balancer via AWS Certificate Manager (ACM)
- Create an HTTP listener on the Application Load Balancer with SSL termination
- Create an HTTPS listener on the Application Load Balancer with SSL termination
- Create an HTTP listener on the Application Load Balancer with SSL pass-through
- Create an HTTPS listener on the Application Load Balancer with SSL pass-through

Overall explanation
Correct option:
"Configure an SSL/TLS certificate on an Application Load Balancer via AWS Certificate Manager (ACM)"
"Create an HTTPS listener on the Application Load Balancer with SSL termination"
An Application load balancer distributes incoming application traffic across multiple targets, such as EC2 instances, in multiple Availability Zones. A listener checks for connection requests from clients, using the protocol and port that you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets. Each rule consists of a priority, one or more actions, and one or more conditions.



via - https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

To use an HTTPS listener, you must deploy at least one SSL/TLS server certificate on your load balancer. You can create an HTTPS listener, which uses encrypted connections (also known as SSL offload). This feature enables traffic encryption between your load balancer and the clients that initiate SSL or TLS sessions. As the EC2 instances are under heavy CPU load, the load balancer will use the server certificate to terminate the front-end connection and then decrypt requests from clients before sending them to the EC2 instances.

Please review this resource to understand how to associate an ACM SSL/TLS certificate with an Application Load Balancer: https://aws.amazon.com/premiumsupport/knowledge-center/associate-acm-certificate-alb-nlb/

Incorrect options:

"Create an HTTPS listener on the Application Load Balancer with SSL pass-through" - If you use an HTTPS listener with SSL pass-through, then the EC2 instances would continue to be under heavy CPU load as they would still need to decrypt the secure traffic at the instance level. Hence this option is incorrect.

"Create an HTTP listener on the Application Load Balancer with SSL termination"

"Create an HTTP listener on the Application Load Balancer with SSL pass-through"

You cannot have an HTTP listener for an Application Load Balancer to support SSL termination or SSL pass-through, so both these options are incorrect.

Question 47

A Company uses a large set of EBS volumes for their fleet of Amazon EC2 instances. As an AWS Certified Developer Associate, your help has been requested to understand the security features of the EBS volumes. The company does not want to build or maintain their own encryption key management infrastructure.

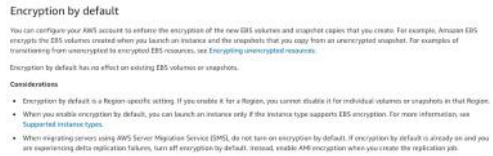
Can you help them understand what works for Amazon EBS encryption? (Select two)

- Encryption by default is a Region-specific setting. If you enable it for a Region, you cannot disable it for individual volumes or snapshots in that Region
- A volume restored from an encrypted snapshot, or a copy of an encrypted snapshot is always encrypted
- You can encrypt an existing unencrypted volume or snapshot by using AWS Key Management Service (KMS) AWS SDKs
- Encryption by default is an AZ specific setting. If you enable it for an AZ, you cannot disable it for individual volumes or snapshots in that AZ
- A snapshot of an encrypted volume can be encrypted or unencrypted

Overall explanation

Correct option:

Encryption by default is a Region-specific setting. If you enable it for a Region, you cannot disable it for individual volumes or snapshots in that Region - You can configure your AWS account to enforce the encryption of the new EBS volumes and snapshot copies that you create. Encryption by default is a Region-specific setting. If you enable it for a Region, you cannot disable it for individual volumes or snapshots in that Region.



Encryption by default

You can configure your AWS account to enforce the encryption of the new EBS volumes and snapshot copies that you create. For example, Amazon EBS encrypts the EBS volumes created when you launch an instance and the snapshots that you copy from an unencrypted snapshot. For examples of transitioning from unencrypted to encrypted EBS resources, see Encrypting unencrypted resources.

Encryption by default has no effect on existing EBS volumes or snapshots.

Considerations

- Encryption by default is a Region-specific setting. If you enable it for a Region, you cannot disable it for individual volumes or snapshots in that Region.
- When you enable encryption by default, you can launch an instance only if the instance type supports EBS encryption. For more information, see Supported instance types.
- When migrating servers using AWS Server Migration Service (SMS), do not turn on encryption by default. If encryption by default is already on and you are experiencing delta replication failures, turn off encryption by default. Instead, enable AMI encryption when you create the replication job.

via - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html

A volume restored from an encrypted snapshot, or a copy of an encrypted snapshot, is always encrypted - By default, the CMK that you selected when creating a volume encrypts the snapshots that you make from the volume and the volumes that you restore from those encrypted snapshots. You cannot remove encryption from an encrypted volume or snapshot, which means that a volume restored from an encrypted snapshot, or a copy of an encrypted snapshot is always encrypted.

Encrypting an empty volume on creation

When you create a new, empty EBS volume, you can encrypt it by enabling encryption for the specific volume creation operation. If you enabled EBS encryption by default, the volume is automatically encrypted. By default, the volume is encrypted to your default key for EBS encryption. Alternatively, you can specify a different symmetric CMK for the specific volume creation operation. The volume is encrypted by the time it is first available, so your data is always secured. For detailed procedures, see Creating an Amazon EBS volume.

By default, the CMK that you selected when creating a volume encrypts the snapshots that you make from the volume and the volumes that you restore from those encrypted snapshots. You cannot remove encryption from an encrypted volume or snapshot, which means that a volume restored from an encrypted snapshot, or a copy of an encrypted snapshot, is always encrypted.

Public snapshots of encrypted volumes are not supported, but you can share an encrypted snapshot with specific accounts. For detailed directions, see Sharing an Amazon EBS snapshot.

via - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html

Incorrect options:

You can encrypt an existing unencrypted volume or snapshot by using AWS Key Management Service (KMS) AWS SDKs - This is an incorrect statement. There is no direct way to encrypt an existing unencrypted volume or snapshot. You can encrypt an unencrypted snapshot by copying and enabling encryption while copying the snapshot. To encrypt an EBS volume, you need to create a snapshot and then encrypt the snapshot as described earlier. From this new encrypted snapshot, you can then create an encrypted volume.

A snapshot of an encrypted volume can be encrypted or unencrypted - This is an incorrect statement. You cannot remove encryption from an encrypted volume or snapshot, which means that a volume restored from an encrypted snapshot, or a copy of an encrypted snapshot is always encrypted.

Encryption by default is an AZ specific setting. If you enable it for an AZ, you cannot disable it for individual volumes or snapshots in that AZ - This is an incorrect statement. Encryption by default is a Region-specific setting. If you enable it for a Region, you cannot disable it for individual volumes or snapshots in that Region.

Question 48

A photo-sharing application manages its EC2 server fleet running behind an Application Load Balancer and the traffic is fronted by a CloudFront distribution. The development team wants to decouple the user authentication process for the application so that the application servers can just focus on the business logic.

As a Developer Associate, which of the following solutions would you recommend to address this use-case with minimal development effort?

- Use Cognito Authentication via Cognito User Pools for your Application Load Balancer
- Use Cognito Authentication via Cognito User Pools for your CloudFront distribution
- Use Cognito Authentication via Cognito Identity Pools for your Application Load Balancer
- Use Cognito Authentication via Cognito Identity Pools for your CloudFront distribution

Overall explanation

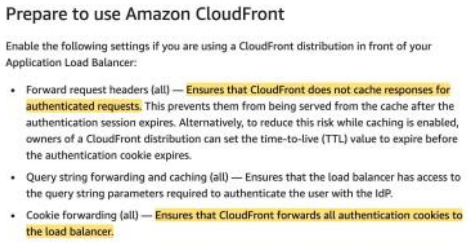
Correct option:

Use Cognito Authentication via Cognito User Pools for your Application Load Balancer

Application Load Balancer can be used to securely authenticate users for accessing your applications. This enables you to offload the work of authenticating users to your load balancer so that your applications can focus on their business logic. You can use Cognito User Pools to authenticate users through well-known social IdPs, such as Amazon, Facebook, or Google, through the user pools supported by Amazon Cognito or through corporate identities, using SAML, LDAP, or Microsoft AD, through the user pools supported by Amazon Cognito. You configure user authentication by creating an authenticate action for one or more listener rules. The authenticate-cognito and authenticate-oidc action types are supported only with HTTPS listeners.

via - https://docs.aws.amazon.com/elasticloadbalancing/latest/application/listener-authenticate-users.html

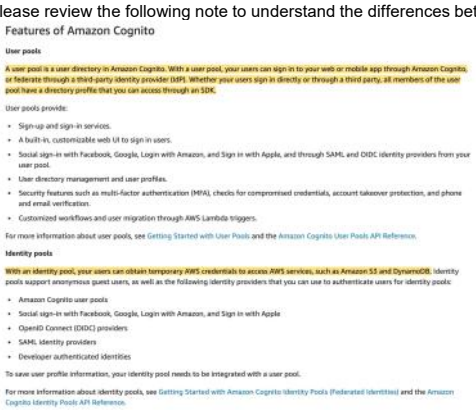
Please make sure that you adhere to the following configurations while using CloudFront distribution in front of your Application Load Balancer:



via - https://docs.aws.amazon.com/elasticloadbalancing/latest/application/listener-authenticate-users.html

Exam Alert:

Please review the following note to understand the differences between Cognito User Pools and Cognito Identity Pools:



via - https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html

Incorrect options:

Use Cognito Authentication via Cognito Identity Pools for your Application Load Balancer - There is no such thing as using Cognito Authentication via Cognito Identity Pools for managing user authentication for the application. Application-specific user authentication can be provided via Cognito User Pools. Amazon Cognito identity pools provide temporary AWS credentials for users who are guests (unauthenticated) and for users who have been authenticated and received a token.

Use Cognito Authentication via Cognito User Pools for your CloudFront distribution - You cannot directly integrate Cognito User Pools with CloudFront distribution as you have to create a separate Lambda@Edge function to accomplish the authentication via Cognito User Pools. This involves additional development effort, so this option is not the best fit for the given use-case.

Use Cognito Authentication via Cognito Identity Pools for your CloudFront distribution - You cannot use Cognito Identity Pools for managing user authentication, so this option is not correct.

Question 49

An analytics company is using Kinesis Data Streams (KDS) to process automobile health-status data from the taxis managed by a taxi ride-hailing service. Multiple consumer applications are using the incoming data streams and the engineers have noticed a performance lag for the data delivery speed between producers and consumers of the data streams.

As a Developer Associate, which of the following options would you suggest for improving the performance for the given use-case?

- Swap out Kinesis Data Streams with SQS FIFO queues
- Use Enhanced Fanout feature of Kinesis Data Streams
- Swap out Kinesis Data Streams with Kinesis Data Firehose
- Swap out Kinesis Data Streams with SQS Standard queues

Overall explanation

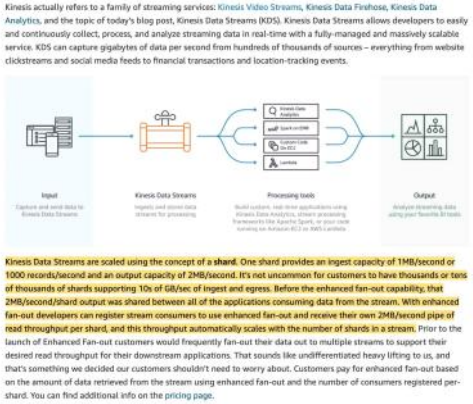
Correct option:

Use Enhanced Fanout feature of Kinesis Data Streams

Amazon Kinesis Data Streams (KDS) is a massively scalable and durable real-time data streaming service. KDS can continuously capture gigabytes of data per second from hundreds of thousands of sources such as website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events.

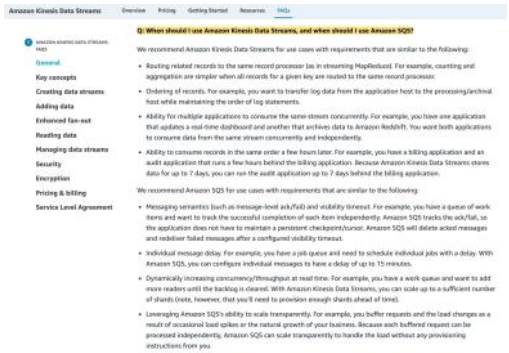
By default, the 2MB/second/shard output is shared between all of the applications consuming data from the stream. You should use enhanced fan-out if you have multiple consumers retrieving data from a stream in parallel. With enhanced fan-out developers can register stream consumers to use enhanced fan-out and receive their own 2MB/second pipe of read throughput per shard, and this throughput automatically scales with the number of shards in a stream.

Kinesis Data Streams Fanout



via - https://aws.amazon.com/blogs/aws/kds-enhanced-fanout/

Incorrect options:
Swap out Kinesis Data Streams with Kinesis Data Firehose - Amazon Kinesis Data Firehose is the easiest way to reliably load streaming data into data lakes, data stores, and analytics tools. It is a fully managed service that automatically scales to match the throughput of your data and requires no ongoing administration. It can also batch, compress, transform, and encrypt the data before loading it, minimizing the amount of storage used at the destination and increasing security. Kinesis Data Firehose can only write to S3, Redshift, Elasticsearch or Splunk. You can't have applications consuming data streams from Kinesis Data Firehose, that's the job of Kinesis Data Streams. Therefore this option is not correct.
Swap out Kinesis Data Streams with SQS Standard queues
Swap out Kinesis Data Streams with SQS FIFO queues
Amazon Simple Queue Service (SQS) is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications. SQS offers two types of message queues. Standard queues offer maximum throughput, best-effort ordering, and at-least-once delivery. SQS FIFO queues are designed to guarantee that messages are processed exactly once, in the exact order that they are sent. As multiple applications are consuming the same stream concurrently, both SQS Standard and SQS FIFO are not the right fit for the given use-case.
Exam Alert:
Please understand the differences between the capabilities of Kinesis Data Streams vs SQS, as you may be asked scenario-based questions on this topic in the exam.



via - https://aws.amazon.com/kinesis/data-streams/faqs/

Question 50

A company has sensitive data stored in an Amazon S3 bucket that is encrypted using AWS Key Management Service (AWS KMS). A developer wants to enforce encryption in transit for all users who have been granted permission to use the S3 GetObject operation across multiple AWS accounts.
Which of the following represents the best solution for this use case?

- Configure a resource-based policy on the S3 bucket to deny access when a request has the condition "aws:SecureTransport": "false"
- Configure a resource-based policy on the S3 bucket to allow access when a request has the condition "aws:SecureTransport": "false"
- Configure a resource-based policy on the KMS key to allow access when a request has the condition "aws:SecureTransport": "false"
- Configure a resource-based policy on the KMS key to deny access when a request has the condition "aws:SecureTransport": "false"

Overall explanation
Correct option:
Configure a resource-based policy on the S3 bucket to deny access when a request has the condition "aws:SecureTransport": "false"
If you want to prevent potential attackers from manipulating network traffic, you can use HTTPS (TLS) to only allow encrypted connections while restricting HTTP requests from accessing your bucket. To determine whether the request is HTTP or HTTPS, use the aws:SecureTransport global condition key in your S3 bucket policy. The aws:SecureTransport condition key checks whether a request was sent by using HTTP.

via - https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html
Incorrect options:
Configure a resource-based policy on the S3 bucket to allow access when a request has the condition "aws:SecureTransport": "false" - This option contradicts the explanation provided above.
Configure a resource-based policy on the KMS key to allow access when a request has the condition "aws:SecureTransport": "false"
Configure a resource-based policy on the KMS key to deny access when a request has the condition "aws:SecureTransport": "false"
Since the use case is about granting permission to use the S3 GetObject operations with encryption in transit, you cannot use a Resource-based policy for KMS. So, both these options are incorrect.

Question 51

As a Developer Associate, you are responsible for the data management of the AWS Kinesis streams at your company. The security team has mandated stricter security requirements by leveraging mechanisms available with the Kinesis Data Streams service that won't require code changes on your end.
Which of the following features meet the given requirements? (Select two)

SSE-C encryption
Encryption in flight with HTTPS endpoint
Client-Side Encryption
KMS encryption for data at rest
Envelope Encryption

Overall explanation
Correct options:
KMS encryption for data at rest
Encryption in flight with HTTPS endpoint
Server-side encryption is a feature in Amazon Kinesis Data Streams that automatically encrypts data before it's at rest by using an AWS KMS customer master key (CMK) you specify. Data is encrypted before it's written to the Kinesis stream storage layer and decrypted after it's retrieved from storage. As a result, your data is encrypted at rest within the Kinesis Data Streams service. Also, the HTTPS protocol ensures that data inflight is encrypted as well.
Incorrect options:
SSE-C encryption - SSE-C is functionality in Amazon S3 where S3 encrypts your data, on your behalf, using keys that you provide. This does not apply for the given use-case.
Client-Side Encryption - This involves code changes, so the option is incorrect.
Envelope Encryption - This involves code changes, so the option is incorrect.